Individual Project
Final Report

ID Number: F231109

Name: Raymond Twum-Barima

Programme: Mechanical Engineering

Module Code: 25WSC500

Project Title: Investigating the efficacy of data-driven techniques and Machine learning algorithms to predict heat transfer characteristics

Abstract

Boiling heat transfer remains difficult to predict accurately due to its transient, spatially heterogeneous nature, which limits the applicability of traditional empirical correlations. This project investigated whether data-driven methods could predict local boiling heat flux from high-speed infrared temperature measurements of a flow boiling setup. A dataset of 17,384,000 pixel-time samples was processed and analysed using Proper Orthogonal Decomposition and Dynamic Mode Decomposition, before being used to train three artificial neural network models of increasing complexity. Model A used raw temperature as a single input, Model B added spatial and temporal gradient features, and Model C mapped the dominant temperature POD modal contributions to their heat flux equivalents. Model C achieved the strongest test performance with R^2= 0.729, $RMSE$= 25,959 W/m^2W/ and MAE= 20,656 W/m^2representing a 69% improvement in explained variance over the linear baseline. The results demonstrate that a reduced-order modal representation captures the dominant temperature heat flux relationship more effectively than raw temperature and spatial inputs, offering a promising framework for local heat flux prediction from transient infrared data.

1 Introduction

Boiling heat transfer is one of the most effective modes of thermal energy removal and is widely used in applications where high heat transfer rates are required, such as power generation, refrigeration, energy conversion systems, electronics cooling, and nuclear thermal management. However, despite its industrial importance, boiling remains difficult to predict accurately as it involves both transport of mass and energy across liquid and vapour phases.

A major challenge in boiling research is that many existing predictive methods are based on empirical or semi-empirical correlations, such as Rohsenow and Kandilkar. While such correlations remain useful, they are usually developed for limited operating and specific conditions, and their performance deteriorates when applied more generally. Review studies[1] have shown that boiling data can exhibit substantial scatter even under apparently similar conditions, and that widely used correlations may not provide sufficiently generalised predictions across a broad dataset. This lack of robustness creates uncertainty in design and limits confidence when transferring correlations between fluids, surface and operating regimes. For engineering applications, if heat flux or wall temperature cannot be predicted reliably, thermal systems may be overdesigned for safety or underperform in practice.

These limitations are particularly relevant when considering transient and spatially resolved boiling behaviour. Traditional correlations often treat the process in an averaged or steady state manner, whereas modern measurement techniques now provide large volumes of high-speed thermal data that reveal strong temporal and spatial variation in surface temperature and heat transfer. Such datasets are difficult to analyse using conventional approaches alone, but they may contain patterns that can be extracted using data-driven methods. Machine learning offers a potential route to address this problem because it can learn nonlinear relationships directly from data, identify important features and generate predictive models. In this study, high-speed thermography measurements provide spatially resolved transient temperature and heat flux information, allowing the boiling process to be examined beyond steady-state averaged behaviour.

This project is motivated by the opportunity provided by high-speed infrared thermography. This project aims to evaluate whether machine learning models can predict the boiling heat transfer process. To achieve this, the objectives are to process the existing experimental boiling dataset, extract key patterns and reduced-order structure from the data, develop and train multiple machine learning models, and compare their predictive performance using standard error metrics.

This report first reviews the background literature on boiling heat transfer modelling and the emerging use of machine learning in this field. It then presents the methodology used to analyse the experimental dataset, extract informative features, and develop machine learning

prediction models. Finally, the results are used to assess whether the transient, high-dimensional boiling data can be modelled effectively using machine learning algorithms.

2 Literature Review

This section will provide an overview of the recent literature in the application of Machine learning (ML) to boiling heat transfer, as well as reviewing traditional modelling of boiling heat transfer, advances in experimental techniques such as High-speed imaging, and data engineering techniques. This will be followed by an analysis of the most beneficial path of inquiry to follow.

2.1 Traditional Modelling Methods

This section reviews traditional modelling techniques used before machine learning techniques. Two groups of traditional modelling were reviewed: empirical correlations and mechanistic models. Early research into boiling behaviour began under Nukiyama, who identified distinct boiling regimes such as nucleate boiling and film boiling.[2], [3]. Empirical correlations are used widely in engineering design due to their ability to provide quick estimates on boiling behaviour, and mechanistic and hydrodynamic models offer deeper physical insight into boiling regimes and critical heat flux. However, their predictive capacity is hampered by simplifying assumptions, a narrow range of validity and sensitivity to surface conditions, fluid properties and operating regime.

2.1.1 Empirical correlations

To understand how empirical correlations the Rohsenow and Kandlikar correlations were reviewed. The Rohsenow correlation is the simpler of the two, developed by W.M Rohsenow in 1952[4] For application in nucleate pool boiling, however, it is not purely empirical since Rohsenow based his correlation on the physical interpretation that heat was transferred mainly from wall to liquid and was enhanced by Bubble agitation and departure from the heated surface. This gives the correlation a physical basis.

$$\frac{c_p(T_w - T_{sat})}{h_{fg} Pr^s} = C_{sf}\left[\frac{q}{\mu h_{fg}}\sqrt{\frac{\sigma}{g(\rho_f - \rho_g)}}\right]^{0.33} \quad [2]$$

The correlation, as seen above, relates heat flux (q) and surface superheat($T_w - T_{sat}$) for a boiling liquid in a pool. The correlation includes properties such as latent heat(h_{fg}), viscosity(μ), specific heat (C_p), surface tension(σ)and the density difference between the liquid and the vapour$(\rho_f - \rho_g)$, together with a constant written as C_{sf} that depends on the particular surface and fluid combination. This is necessary because boiling behaviour is strongly dependent on the heating surface conditions and how they interact with different liquids.

Rohsenow reported that, for a given clean surface-fluid combination, using the above correlation for nucleate pool boiling reproduces the experimental data with an accuracy of

roughly 20%[4]. It provides a simple way of estimating nucleate boiling performance. However, the correlation has some significant drawbacks, firstly it is intended for use specifically within the nucleate boiling regime, it is not a general boiling model, secondly it depends on the constant C_{sf} meaning its accuracy depends heavily on whether there is an appropriate constant for the surface fluid pair being modelled, thirdly the correlation depends on exponents that should not be view as physical truths but arbitrary values that seem to correspond to the data, lastly with specific importance to the project, Rohsenow is a fundamentally a averaged nucleate pool boiling correlation, where as our dataset contains transient and spatially resolved temperature and heat flux fields in a flow boiling setup.

The central Idea within Kandlikar's[5] model was that flow boiling is not governed by a single physical mechanism. Therefore, the total two-phase heat transfer coefficient can be separated into convective boiling and nucleate boiling contributions.

$$\frac{h_{TP}}{h_l} = C_1 Co^{C_2} + C_3 Bo^{C_4} F_{fl}$$

In the paper, the correlation is written in dimensionless form relative to a liquid only heat transfer coefficient (h_l) which is written above. The main parameters are the convection number *(Co)*, the boiling number (Bo) and a fluid-dependent constant (F_{fl}). For horizontal cases, a Froude number correction is also included to account for stratification effects. Kandlikar argues that a major requirement of a general correlation is the introduction of the fluid-dependent parameter. F_{fl} this allows the correlation to extend to new fluids. This is a useful strength because it makes the correlation adaptable, but also highlights that it is still not universal, depending on a fitted fluid-specific parameter.

In terms of performance, Kandlikar reports that the final correlation gives a mean deviation of 15.9% [5]and 18.8%[5] for all refrigerant data combined, compared to other well-known correlations, the Kandilkar correlation gives the lowest mean deviation.

Kandilkar is more general than simple pool-boiling correlations such as Rohsenow because it was developed for both convective and nucleate boiling contributions. Despite this, it remains a correlation-based model in which complex boiling behaviour is represented through simplified dimensionless groups and fitted parameters. It outputs an averaged heat transfer coefficient rather than a local description of boiling behaviour; for this reason, it is still not well suited to the transient, spatially resolved thermal data used in this study.

2.1.2 Mechanistic models

A key mechanistic approach to boiling heat transfer is Zuber's hydrodynamic instability model[6]. Unlike empirical correlations, which relate boiling performance to fitted parameters, Zuber interpreted the critical heat flux as a hydrodynamic limit. He proposed that sufficiently high heat flux caused vapour generation above the heater to become so intense that the upward vapour motion disrupts the downward replenishment of liquid to the surface. In greater detail, vapour escapes from the heater in the form of jets or columns, while liquid

occupies the region between them and attempts to move back towards the surface. Critical Heat Flux is reached when these vapour structures become unstable, preventing effective liquid rewetting of the heater. In this sense, Zuber's model is more physically explanatory than empirical correlations such as Rohsenow and Kandlikar, which are primarily intended for engineering prediction rather than mechanistic interpretation.

For saturated pool boiling from a horizontal surface, Zuber's analysis leads to the well-known critical heat flux relation.

$$q''_{CHF} = 0.131 h_{fg} \rho_v^{1/2} [\sigma g(\rho_l - \rho_v)]^{1/4}$$

Where h_{fg} is the latent heat of vaporisation, ρ_v is the vapour density, ρ_l is the liquid density, σ is the surface tension, and g is the gravitational acceleration.

Despite its physical importance, Zuber's model has limitations. The model is based on an idealised vapour-jet structure and is most applicable to saturated pool boiling on large horizontal surfaces. Later work has shown that classical CHF models, including Zuber's, generally provide estimates within an uncertainty range rather than universally accurate predictions for all heater geometries, pressures, and surface conditions. For example, Ertunc[7] noted that such classical correlations remain approximate baselines, while an ANN-based predictor in the same study achieved a correlation coefficient of 0.996[7], a mean relative error of 8.81%[7], and an RMSE of 0.97 W/cm²[7].

2.1.3 Limitations of Traditional Methods

Taken as a whole, Rohsenow, Kandilkar and Zuber highlight the main strengths of traditional boiling models: they are interpretable, physically motivated to different degrees, and useful for engineering estimation within their intended regimes. However, they also share an important limitation for the current project they were developed for, which averaged boiling quantities under restricted conditions, whereas the project's dataset consists of transient, spatially resolved thermal fields. They are not naturally suited to extracting hidden structure or predicting local heat transfer behaviour. This is the gap that motivates the use of reduced-order methods and machine learning in the present work.

2.2 Advances in Experimental Techniques

Our understanding of boiling heat transfer has been heavily linked to the development of improved experimental data. Traditional measurement methods, such as thermocouples and local resistive temperature sensors, often suffer from limited spatial and temporal resolution, making them unsuitable for capturing fast local processes such as bubble nucleation, growth and departure. Similarly, high-speed optical imaging alone can become difficult to interpret in developed nucleate boiling, where large numbers of bubbles obscure individual events on the surface.

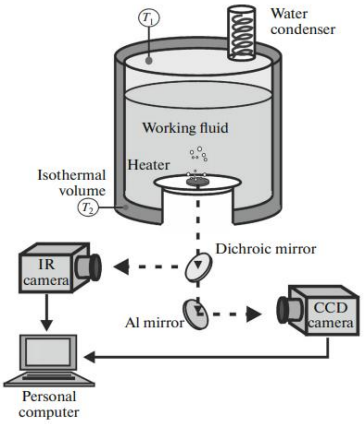


Figure 1 - Diagram of experiment in Surtaev's paper [8]

A significant development in this area is the use of high-speed infrared thermography, Surtaev et al.[8]. Describe a method for studying both local and spatially averaged boiling characteristics using a thin film heater combined with a high-speed IR camera and simultaneous optical recording, as seen in Figure 1. In their experiments on saturated ethanol, an indium-tin oxide thin film heater deposited on a sapphire substrate allowed researchers to measure the non-stationary temperature field on the heater surface while visually recording bubble growth and departure. This approach made it possible to analyse temperature evolution beneath individual bubbles.

The results reported by Surtaev et al demonstrate the value of this technique for resolving the boiling processes. By tracking the changes in temperature and the corresponding bubble dynamics, they were able to characterise the full thermal cycle beneath the active nucleation sites. They also showed that nucleation site density increased approximately linearly with heat flux.

These advances are important because they provide the kind of spatially and temporally resolved surface-temperature data that older methods could not capture reliably. More broadly, they also create new opportunities for data-driven analysis, since modern machine learning methods require large amounts of reliable local experimental data to identify complex relationships between temperature fields, nucleation behaviour, and heat transfer response

2.3 Machine Learning in Boiling Heat Transfer

In recent years, machine learning has started to receive much more attention in boiling heat transfer. The main reason for this is that boiling is a very complex process that involves many interacting variables across different length and time scales. Upadhyay et al[1] points that this is one of the main motivations for using machine learning in boiling research, since ML can analyse large datasets and identify patterns that are difficult to capture using traditional approaches.

A useful point made by the recent review papers is that machine learning in boiling is not just about fitting another correlation. Upadhyay et al[1] shows that the field already includes several different directions, including regression-based predictions, post-processing of boiling data, mechanistic understanding, and computational studies. They also explain that the success of ML depends on a number of things, especially the quality of the dataset, the choice of features, the selected algorithm, and the performance metrics used for validation. In addition, they note that one of the common weaknesses of ML models is that they may not generalise well beyond the range of training data. This is important in boiling because experimental datasets are often scattered, incomplete, and collected under different conditions.

Chu et al.[9] make a similar argument, but in a shorter and more application-focused way. Their review states that machine learning is mainly being used to predict important boiling quantities such as heat flux, heat transfer coefficient (HTC), and critical heat flux (CHF), since these are often difficult to describe accurately using conventional mathematical correlations alone. They also note that some recent studies have used ML together with image processing to identify boiling states and even predict heat flux from bubble morphology, which is useful because it creates the possibility of non-contact measurements during boiling experiments. At the same time, they are careful to point out that ML still has limitations, especially the risk of overfitting when the training dataset is too small or not representative enough.

Another reason ML is attractive in boiling is that it can deal better with nonlinearity than traditional approaches. Upadhyay et al.[1] emphasise that the heat transfer rate in boiling is not controlled by a single variable, but by a complicated interaction between surface properties, fluid properties, operating conditions, and bubble dynamics. Because of this, it is often difficult to represent the process using a simple equation form. They argue that ML methods are useful here because they can handle nonlinear relationships more effectively, and they can also help simplify large datasets through feature selection and dimensionality reduction. The review also highlights that ML can work with different forms of data, such as bubble images, thermal maps, and boiling acoustics, which opens new possibilities for real-time analysis, visualisation, and control.

The literature also shows that ML is already being used to improve prediction in both pool boiling and flow boiling. For example, the review by Upadhyay et al. summarises studies where machine learning models were used to predict flow boiling HTC in mini and microchannels using very large datasets. In one of the examples listed in the review, feature optimisation methods such as Pearson correlation and mutual information were used to reduce the number of inputs, and the resulting ANN model achieved a MAPE of 8.48%, outperforming both earlier ML models and universal correlations.

More recent work also shows how ML is being applied to specific enhanced boiling surfaces. A recent study[10] on micropillar surfaces used Bayesian optimisation to tune four machine learning models to establish a mapping between heat transfer coefficient and the key surface

descriptors. Even though this is a more specialised case, it is useful because it shows the direction the field is moving in. Instead of relying only on a single empirical correlation, researchers are starting to use ML to link boiling performance directly to structured surface features in a more systematic way.

Overall, the current Machine Learning literature suggests that machine learning is becoming an important tool in boiling heat transfer because it can handle nonlinear behaviour, large datasets, and richer experimental inputs better than many traditional methods. It has already been applied to predicting heat flux, HTC, CHF, pressure drop, and boiling state classification, and it is also proving useful for image-based analysis and optimisation of enhanced surfaces. However, the literature also shows that the main challenges are still data quality, generalisation, and interpretability. For the present project, this is particularly relevant because the available data are highly resolved in both space and time. This means ML is not only useful as a predictive tool, but also as a way of identifying patterns in thermal field data that may not be obvious from conventional boiling correlations alone.

2.4 Dimensionality reduction

Dimensionality reduction is important in boiling heat transfer because spatially resolved temperature measurements generate very large datasets that are difficult to analyse directly. Recent review literature highlights dimensionality reduction as a useful way to simplify complex boiling data while retaining the most important structures, making later prediction and interpretation more manageable. In machine learning studies, this is often done using methods such as principal component analysis, which transform the original variables into a smaller set of dominant features.

In the present work, a similar role is played by Proper Orthogonal Decomposition (POD). POD reduces the dimensionality of the temperature field by identifying the dominant spatial modes that contain most of the variance in the data. Although the study by Hajisharifi et al[11]. It is not a boiling paper; still, it is a useful method because it shows how POD-based reduced-order modelling can reconstruct full temperature fields efficiently from sparse data. This supports the use of POD here as a way of compressing high-dimensional thermal data into a smaller number of physically meaningful structures before further analysis

2.5 Discussion and Project Aims

The literature reviewed in the previous sections provides a clear justification for the direction of this project. Traditional boiling models, including empirical correlations and mechanistic approaches, remain useful because they offer either practical engineering prediction or physical insight. However, they are also limited by fitted constants, simplifying assumptions, and narrow ranges of validity. This is especially important when dealing with transient and spatially resolved boiling behaviour, where averaged or steady-state correlations are often not sufficient.

Recent advances in experimental techniques, particularly high-speed infrared thermography, have made it possible to capture local temperature evolution, nucleation behaviour, and transient bubble-related thermal processes in much greater detail. At the same time, recent review papers show that machine learning is becoming increasingly relevant in boiling heat transfer because boiling data are high-dimensional, noisy, and strongly nonlinear. These studies suggest that ML can improve the prediction of quantities such as heat flux, heat transfer coefficient, and critical heat flux, but they also emphasise that success depends strongly on dataset quality, feature selection, and model generalisation.

While a substantial part of the ML boiling literature has focused on extracting information from bubble images, the present project instead uses infrared-derived thermal fields as the predictive basis. This is meaningful as bubble morphology can correlate with heat transfer, but it remains a surrogate for the underlying wall process, whereas IR measurements provide direct access to the transient thermal data. The approach, therefore, aims to predict heat flux from data that are closer to the heat-transfer mechanism itself.

This directly motivates the present project. As defined in the original project description and refined in the Gateway 2 presentation, the aim is to investigate whether machine learning models can predict local boiling heat transfer behaviour using experimentally derived temperature and heat-flux data. The main objectives are to process and visualise the experimental dataset, implement baseline empirical comparisons, apply POD to identify dominant thermal modes, train supervised ML models to predict local heat flux, and evaluate their performance using standard error metrics such as RMSE and correlation-based measures.

3 Methodology

3.1 Method overview

This project used an experimental dataset obtained via high-speed infrared thermography of a flow-boiling setup. The dataset contained transient temperature measurements over time and was processed in MATLAB before being used for reduced-order analysis and machine learning. First, the raw temperature data were cropped to isolate the region of interest. The data was visualised to gain a broader understanding of the dataset. Next, dimensionality reduction techniques are applied to identify the dominant spatial and temporal patterns within the data. The reduced and processed data are used to train a set of machine learning models for Heat flux prediction. Finally, the models are evaluated and compared with standard performance metrics.

3.2 Experimental dataset

An infrared high-speed imaging camera was used to capture spatial and temporal thermal fields of a flow boiling setup. The temperature data was stored as a four-dimensional matrix

of size [51 x 116 x 11 x 4,043] corresponding to the spatial coordinates n_y, n_x, n_z , and the number of time steps n_t. A schematic representation of the dataset is shown in the Appendix. 8.4.2. At each timestep, the dataset contains temperature measurements over a three-dimensional domain, giving a total of 65,076 temperature values per timestep. Similarly, the heat flux data were stored in a three-dimensional matrix of size [51 x 116 x 4,043] corresponding to the spatial coordinates n_y, n_x, and the number of time steps n_t. There is no n_z, dimension since heat flux is measured only at the heater's surface. The 4043 snapshots provide a time-resolved record of thermal evolution of the boiling process over a 1s time period. Each pixel is 0.24mm × 0.22mm ($n_y \times n_x$).

3.3 Data preprocessing

3.3.1 Extract heater surface temperature

Before the dataset could be used for reduced-order analysis and machine learning modelling, a few preprocessing steps were applied to ensure consistency and remove unwanted edge effects. First, the temperature matrix was squeezed along the n_z dimension to extract the temperature field at the heater surface, reducing the data from a four-dimensional matrix to a three-dimensional matrix of size [51 x 116 x 4,043]

3.3.2 Spatial and Temporal Cropping

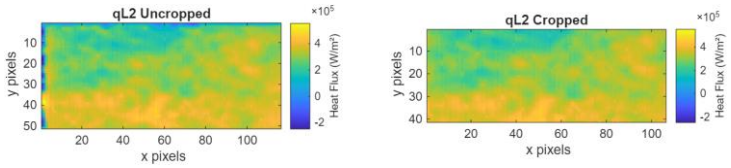


Figure 2 - uncropped and cropped Heat flux field

Next, both temperature and heat flux matrix were cropped by 5 pixels along each boundary to remove the edge distortions at the heater surface; the dimensions of both datasets were reduced to [41 x 106 x 4,043]. Lastly, the last 43 time frames were removed so that a glitch could be removed that showed the heat flux to be zero. The final matrix size was [41 x 106 x 4,000]. This produced a total dataset of 17,384,000 pixel-time samples for both heat flux and temperature datasets.

3.3.3 Preparation for POD/DMD

To prepare the temperature and heat flux datasets for Proper Orthogonal Decomposition (POD)and the Dynamic Mode Decomposition (DMD), each three-dimensional field was reshaped into a 2D matrix of the number of pixels in each frame. This produced a data matrix of size [n_{pix} x 4,000], where n_{pix} = n_y X n_x. The data were then mean-centred to isolate the fluctuating component of the signal. This was carried out according to

$$X_c = X - X_{mean} \qquad (1)$$

Where X is the original data matrix, X_{mean} is the temporal mean field and X_c is the centred matrix used in the subsequent reduced-order analysis. The code for this is in the Appendix section. 8.1.2.

3.3.4 Validation

A preliminary validation step was to be carried out before applying POD and DMD, to confirm that the processed data remained physically consistent. The spatially averaged surface temperature and spatially averaged heat flux were plotted over time and plotted together as shown in the Appendix 8.3.1. The two signals showed an inverse relationship, with increases in mean flux generally corresponding to a reduction in mean temperature. This trend is consistent with the expected physical behaviour of the system and therefore provided confidence that the preprocessing steps had preserved the main thermal characteristics of the dataset.

3.4 Proper Orthogonal Decomposition

Proper Orthogonal Decomposition (POD) is a reduced-order modelling technique used to simplify high-dimensional datasets by identifying the dominant structures within them. In the present study, POD was applied to the temperature and heat flux mean-subtracted fields to extract a small number of POD modes that retain most of the variance in the data. Each mode is associated with a temporal coefficient. This provides a physically meaningful reduced-order description of the boiling temperature and heat field while preserving the main thermal structures relevant to subsequent analysis and prediction.

3.4.1 Singular value decomposition

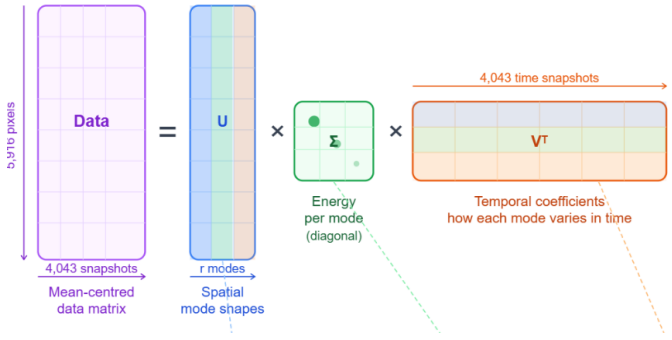


Figure 3 - schematic representation of the mean-centred data matrix [AI generated by Claude Sonnet]

The main process behind POD is the singular value decomposition, a matrix factorisation process that can be applied to any complex matrix. The mean-centred data matrix X_c was decomposed using singular value decomposition such that

$$X_c = U\,\Sigma\,V^T$$

In this formulation, the columns of U represent the spatial modes of the system, Σ contains the singular values that indicate the relative energy or variance captured by each mode, and row V^T describes the temporal evolution of these modes. The first few modes therefore capture the dominant thermal structures within the boiling dataset while subsequent modes represent progressively weaker and smaller-scale behaviour. This decomposition enables the high-dimensional temperature and heat flux fields to be represented in a reduced order form. The MATLAB code for this can be found in the section. 8.1.3

3.4.2 Modes Analysis

To quantify the contribution of each POD mode, the singular values obtained from the diagonal of Σ were used to calculate the relative modal energy for the i-th mode, the energy fraction was defined as

$$E_i = \frac{\sigma_i^2}{\Sigma_{j=1}^{r}\sigma_j^2} \qquad E_{cum(k)} = \frac{\Sigma_{i=1}^{k}\sigma_i^2}{\Sigma_{j=k}^{k}\sigma_j^2} \qquad (2)$$

Where σ_i is the i-th singular value and r is the total number of retained modes. This provides a measure of the proportion of the total dataset variance captured by each mode. The cumulative energy captured by the first k modes were then calculated. The MATLAB code can be seen in 8.1.3. The cumulative energy distribution was used to assess how many modes were required to retain the dominant thermal structures within the dataset.

The columns of U were reshaped into two-dimensional fields to visualise the spatial POD modes. The contribution of the i-th POD mode to the temperature field at each spatial location X and time t is defined as the rank-1 field.

$$C_i(x,t) \;=\; \Phi(x)\Sigma_i v_i(t)$$

where $\Phi(x)$ is the i-th column of U, Σ_i is the corresponding singular value and $v_i(t)$ is the i-th row of V^T. The full mean-subtracted field is recovered by summing contributions across all retained modes

$$X_c(x,t) \;\approx\; \sum_{i=1}^{k} C_i(x,t) \qquad (3)$$

the temporal coefficients were obtained from the singular values and right singular vectors according to

$$A = \; \Sigma\, V^T$$

where each row of A corresponds to the time-dependent amplitude of one POD mode. These coefficients describe how strongly each spatial mode contributes at each time step. The code for the plotting of both the spatial modes and temporal coefficients can be seen in the Appendix section. 8.1.4.

3.5 Dynamic Mode Decomposition

Dynamic Mode Decomposition (DMD) is the first data-driven modelling technique used in this study. Unlike neural network-based methods, it is founded on the Koopman operator, an infinite-dimensional linear operator that provides a way of representing nonlinear dynamical systems, such as the flow boiling process examined here, in a linear fashion.

DMD assumes that two snapshot matrices are related through a linear operator:

$$X_2 = A\,X_1$$

Where A is the linear operator between matrices X_2 and X_1, DMD was applied to the mean-subtracted data matrix X_c as described in the section 3.3.3. The first step is to create snapshot pairs from X_c such that matrix X_2 is composed of data values from timesteps t_2 to t_{4043}, and matrix X_1 is composed of data values from timesteps t_1 to t_{4042} so that they are offset from each other, as illustrated in the Appendix 8.4.3.

Next, SVD is performed on the matrix. x_1 like the process described in 0After this, the number of modes to capture 99% of the energy is computed, and this is then used to truncate the matrix X_1This reduces noise and makes further steps less computationally expensive.

$$X_1 = U_r \Sigma_r V_r^T$$

The reduced order linear operator is then computed as

$$\tilde{A} = U_r^T X_2 V_r \Sigma_r^{-1}$$

Where $\tilde{A}$ is the reduced order linear operator, X_2 is the snapshot pair matrix, U_r^T is the transpose of the truncated spatial modes matrix of X_1, V_r are the truncated temporal coefficients of X_1 and Σ_r^{-1} is the inverse of the singular values matrix of X_1 inversed and truncated.

From the $\tilde{A}$ matrix the eigenvalues and eigenvectors were extracted, and used them to construct the DMD modes

$$W\,, \Lambda = eig(\tilde{A}) \qquad \phi = X_2 V_r \Sigma_r^{-1} W$$

Where W are the eigenvectors and Λ are the eigenvalues of the $\widetilde{\mathrm{A}}$, ϕ are the DMD modes with each column in the matrix representing a DMD mode.

$$\omega = \frac{\ln(\lambda)}{\Delta T}$$

Where λ are the eigenvalues, ΔT is the timestep and ω are the frequencies, which allowed the plotting of how each mode oscillates over time. A set of weights b_k are computed so that the DMD modes combine to reconstruct the initial conditions

$$x_1 = X_c^{(1)} \quad b = \Phi^{\dagger} x_1$$

where $X_c^{(1)}$ is the mean-subtracted matrix at the first timestep, and $\Phi^{\dagger}$ is the pseudoinverse of ϕ the DMD modes. The amplitude of each mode at each time step is calculated using this equation

$$a_k(t) = b_k e^{\omega t}$$

with $a_k(t)$ being the amplitude of k DMD mode at time t. The code for this section is in the Appendix 8.1.5

$$x(t) \approx \sum_{k=1}^{r} \phi_k \, a_k(t) \qquad (4)$$

using equation 4, the heat flux field can be forecasted using the DMD formulation above. In this project, DMD was applied to the heat flux data, with the reduced-order linear operator $\tilde{A}$ trained using the first 2800 time frames, the resulting model was then used to forecast the evolution of the heat flux field over the subsequent 1200 time frames. The code is shown in Appendix section 8.1.5.

3.6 Machine learning models

The fundamental building block for a neural network is a neuron. It receives an input(number), then multiplies it by a weight, sums up the inputs and adds a bias. This then goes through an activation function. This can be described mathematically as

$$output = f(w_1 x_1 + w_2 x_2 + \ldots\ldots + w_n x_n + b)$$

where x are the inputs, w are the weights, b the bias and f is the activation function.

Without the activation function, the network could only learn linear relationships. Activation functions are mathematical functions applied to each neuron that determine how strongly a neuron's signal fires.

The data flows through the neural networks in what is called 'forward pass' at each layer, the inputs are transformed through the weights until a final prediction is produced. The prediction is compared to the real value, and a loss function quantifies the error.

After the losses have been quantified, the weights that contributed to the error need to be adjusted. Calculus through the chain rule is used to compute the partial derivative of the loss with respect to each weight, resulting in a gradient for each weight informing whether to increase or decrease the weights and by how much.

The new weight is then computed as

$$w_{new} = w_{old} - \eta \times \nabla L$$

Where w are the weights, η is the learning rate and ∇L is the gradient of the loss, so as the model iterates through each forward pass and back propagation, the network gradually learns the system's behaviour.

3.7 Model training and validation

The neural network models in this project followed those shown in the section 3.6 for the input and output data. The data was then divided 70:30 into training and testing datasets. The hyperparameters were selected using Bayesian optimisation, a method of finding a global optimum. Lastly, the predictive performance was assessed through standard error metrics.

3.7.1 Feature selection

The initial model, model A, was trained using just one input, using raw temperature to predict the local heat flux. This was used as a baseline to establish whether temperature alone was sufficient to predict heat flux. Model B was inspired by the Heat flux equation.

$$q = -k \frac{dT}{dx} \quad (5)$$

and used 4 input features: temperature, the temporal temperature gradient $\frac{dT}{dt}$ and the spatial temperature gradients $\frac{dT}{dx}, \frac{dT}{dy}$ Lastly, model C was trained using the highest 5 temperature POD modal contributions to map to the heat flux POD modal contributions, and then to reconstruct the heat flux field from the predicted modal contributions. A schematic of model C has been provided in the Appendix section 8.4.1 The order of complexity increased at each level, allowing for the evaluation of the effect of feature selection on the accuracy of the model. The code for the feature construction of Model C and B is in Appendix sections 8.2.1 and 8.2.2.

3.7.2 Hyperparameter selection

Due to the computational expense of running a neural network, it was deemed unfeasible to spend large amounts of time trying every combination of hyperparameters for each model. Instead, a technique called Bayesian optimisation was implemented. It worked by using a smaller subset of the dataset. It then uses a different combination of the selected range of hyperparameters at each trial. It puts the good configurations and bad configurations into separate piles and determines what the good configurations have in common. With the next trial, it picks two combinations, one resembling the good pile and one resembling the bad and learns from each iteration. The code is in Appendix section 8.2.3

3.8 Performance metrics

To examine the accuracy of each neural network, a set of error metrics was used to identify differing characteristics of the model. R^2 the coefficient of determination, the mean absolute error (MAE) and the root mean squared error (RMSE). The predicted heat flux value was compared to the true heat flux value, to calculate as follows

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} \quad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \quad \text{RMSE} = \sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$$

y_i is the true value, $\hat{y}_i$ is the predicted value, $\hat{y}_i$ is the mean value and $\frac{1}{n}$ is the normalisation. The R^2 value measures how much of the variance in the data the model explains with an R^2 value of 1 being a perfect prediction and 0 being no better than predicting the mean. MAE is the average size of the error, and RMSE is similar but penalises outliers harsher. Each error metric was calculated on both the training set and testing set to evaluate the ability of the model to perform beyond its training data. The code for the error metrics is written in Appendix section 8.2.5.

3.9 Use of AI

AI assistance was used during code development in the form of large language model tools, primarily ChatGPT but also Claude sonnet, which was used to generate schematic illustrations, but mainly were used to support drafting, debugging and explanation of MATLAB and Python code for POD DMD, feature construction, Bayesian optimisation and plotting routines. Typical prompts asked for help understanding algorithm structure, plotting and handling errors. The generated output was not used unedited; they were adapted to the project dataset and methodology by the Author. All design choices, such as algorithm, input features selection and the use of Bayesian optimisation, the interpretation of results, and validation, remained the author's own work.

Large language model tools, primarily ChatGPT and Claude Sonnet, were used in a limited capacity to assist with grammar checking and improving the clarity of technical explanations. All content, arguments, and technical interpretations remain entirely the author's own work, with no sections reproduced entirely from AI-generated output.

4 Results and discussions

4.1 Dataset behaviours and preprocessing observations

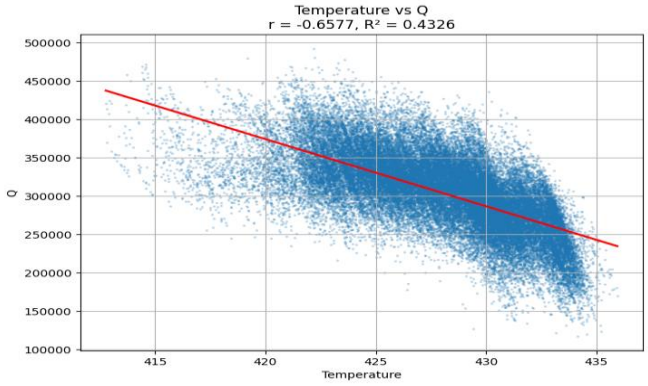


Figure 4 - A scatter graph of temperature and heat flux

The scatter plot shown in Figure 4 used a random sample of 50000 pixel-time points of the processed temperature and heat flux data to show the relationship between temperature and heat flux across the dataset. The Pearson correlation coefficient of -0.6577 indicates a

moderate inverse relationship. While the R^2 = 0.4326 shows that temperature alone explains 43% of the variance in the heat flux. Any prediction algorithm should be compared to R^2 = 0.4326 to evaluate if it has learned non-linear behaviour. Although this confirms that temperature is physically informative, the scatter suggests that the relationship is not purely linear and that additional spatial and temporal, or modal information may be required.

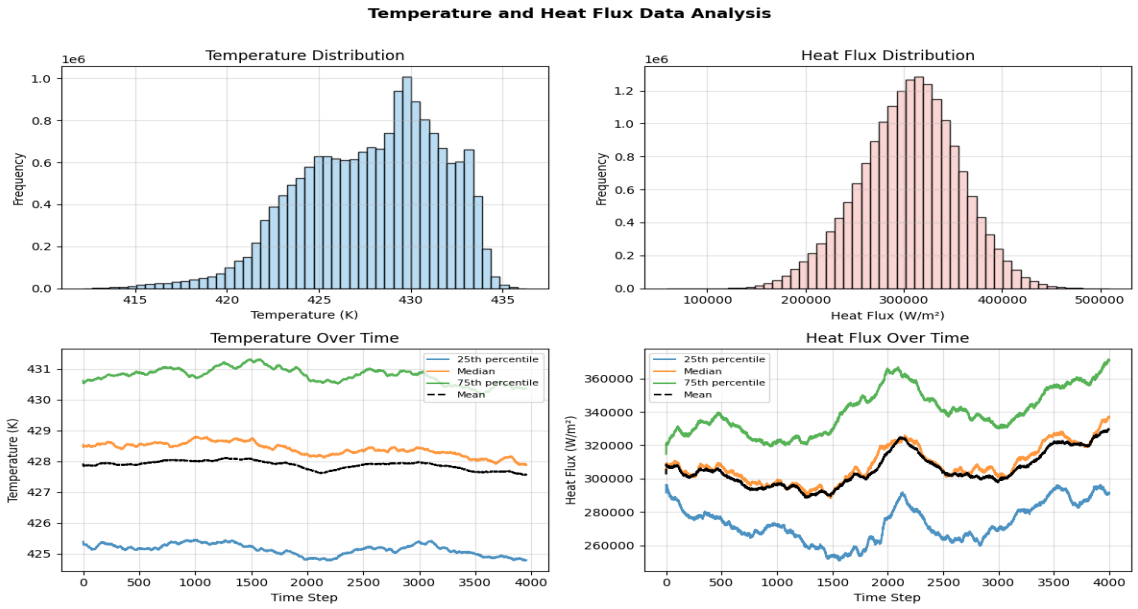


Figure 5 - statistical summary of the temperature and heat flux datasets, showing overall distributions and temporal evolution of the 25th,50th,75th percentile and the mean

Figure 5 summarises the statistical behaviour of the temperature and heat flux dataset. The histogram of temperature and the temperature percentile highlight the concentrated nature of the data, with the 25th percentile hovering around 425K and the 75th percentile around 431K, meaning 50% of the data is within a 5 kelvin range. The temperature percentile curves remain close together and are smooth, implying the temperature field changes gradually and retains a stable structure. The histogram distribution is not perfectly symmetric, suggesting that different thermal conditions may be contributing.

On the other hand, the heat flux histogram and percentile graph paint a different picture. The heat flux distribution has a significantly broader range. A graph highlighting the interquartile range (IQR) has been provided in the Appendix 8.3.2. The IQR of the heat flux ranges from 20,000 w/m^2 to 90,000 w/m^2illustrating how the heat flux data is more spatially and temporally heterogeneous.

The dataset clearly reveals that temperature is physically significant in predicting heat flux, but the higher complexity and greater variation displayed by the heat flux data indicate that temperature alone is unlikely to capture all the underlying behaviour, and justifies the later use of spatial and modal features to train machine learning algorithms.

The dataset was split into a training and test set, 70:30, for the evaluation of the model. The heat flux split was plotted in a graph in the Appendix 8.3.3. The train and test distribution overlap substantially, so the split remains similar, but the test set has a slightly higher mean, with a train set mean of 302261.55 W/m^2 and test set mean of 313161.48W/m^2. This makes the test problem more challenging.

4.2 POD Results

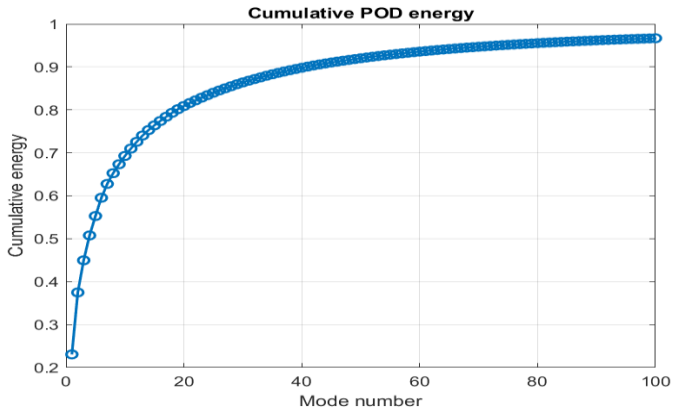


Figure 6 - Graph of the cumulative temperature POD energy

Figure 6 is a graph of the cumulative temperature POD energy on the mean-subtracted matrix X_c as calculated with equation 2 and implemented in Appendix 8.1.3. It highlights how much energy or variance in the mean-subtracted data or fluctuating component is explained by the first k modes. The rapid rise in the cumulative energy indicates the first few modes account for the majority of the variation, with the first 5 modes capturing 55.28% of the fluctuation energy, the first 10, 69.27%, and the first 40, 90%. However, the gradual tail beyond the first few modes shows that smaller-scale transient features still contribute meaningfully to the fluctuation energy.

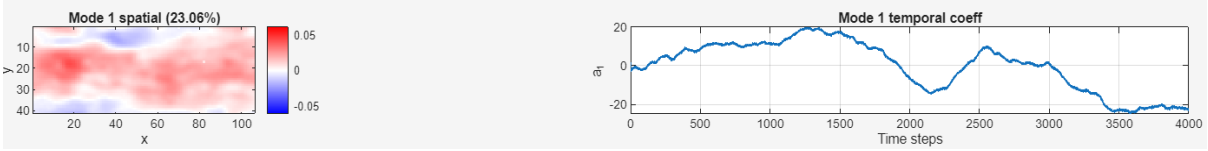
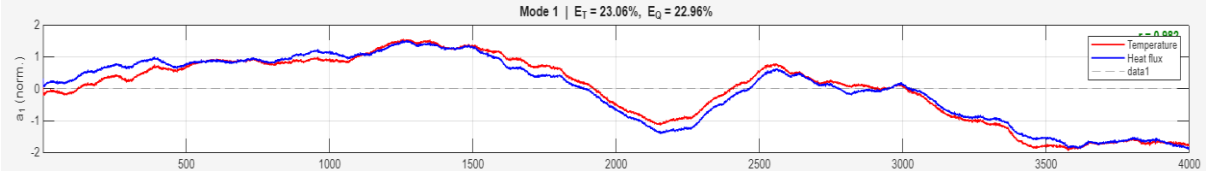


Figure 7 - Spatial Mode 1(left) and Temporal coefficient mode 1 (right)

Refining the POD analysis, the first 10 POD modes U , were plotted together with their temporal coefficients $A = SV^T$, capturing 69.27% of the fluctuation energy. The spatial plot reveals the dominant pattern of deviation from the mean temperature field. Red and blue regions indicate positive and negative signed deviations from the mean-subtracted field. While the Temporal coefficient describes how strong the strength and sign of this pattern is over time.

Figure 7 illustrate the spatial plot and temporal coefficient of mode 1, with the rest of the modes plotted in the Appendix 8.3.4. From these, it is possible to determine the characteristics of each mode. Mode 1 shows a large coherent structure across much of the domain, indicating that it represents the dominant large-scale deviation from the mean

temperature field. The temporal coefficients change slowly with the largest amplitude of all the modes, indicating that it is the main governing pattern behind the temperature fluctuations. Mode 2 shows a strong contrast between the different regions of the surface, the top and bottom, suggesting it describes a redistribution of thermal activity. Its temporal



coefficient becomes strongly positive in the middle of the time interval. The higher order modes have more fragmented and localised spatial patterns, and their temporal coefficient become more oscillatory. indicating they represent small-scale transient variations.

Figure 8 - Mode 1 heat flux and temperature coefficients overlayed

A similar analysis was performed on the mean-subtracted heat flux field in the Appendix 8.3.5 and their temporal coefficients. The temperature modes were paired with their corresponding heat flux modes, and by plotting their normalised temporal coefficients together, it could be identified if they were physically linked. These plots are shown in the Appendix 8.3.7, while the graph of the mode 1 pair is shown in Figure 8. The correlation between the coefficients are all high, with mode 1: r =0.98, mode 2: r = 0.97, mode 3: r = 0.80, mode 4: r = 0.81, mode 5: r = 0.96. These results suggest that the POD is extracting meaningful physics and that predicting heat flux modal coefficients from temperature modal coefficients is a sensible reduced-order modelling strategy.

4.3 DMD Results

In section 3.5 the methodology behind the DMD is illustrated, with implementation details in the Appendix 8.1.5. The accuracy of this prediction method was evaluated using the error metrics mentioned in the section 3.8. These error metrics are time-resolved in the Appendix 8.3.8. The error results in the test region (timesteps after 2800), have been averaged, giving a R^2 = 0.3844, MAE = 30,525.3856 w/m^2 and RMSE =39,144.2773 w/m^2.

The graph in the Appendix 8.3.8 illustrates the true spatial mean heat flux overlayed with the DMD mean forecast in the test region. The prediction captures the overall upwards trend in the mean heat flux; the forecast is noticeably smoother and fails to reproduce the later rise in the mean heat flux. The metrics shown in Appendix 8.1.5 mirror these results, with early results in MAE and RMSE being relatively low, but then suddenly rising and then reaching a plateau.

Overall, the results suggest that DMD provides a reasonable short-horizon prediction of the broad mean trend, but accuracy deteriorates quickly as the forecast horizon increases and is unable to reproduce the full transient behaviour.

4.4 Artificial Neural Networks Results

4.4.1 Model A

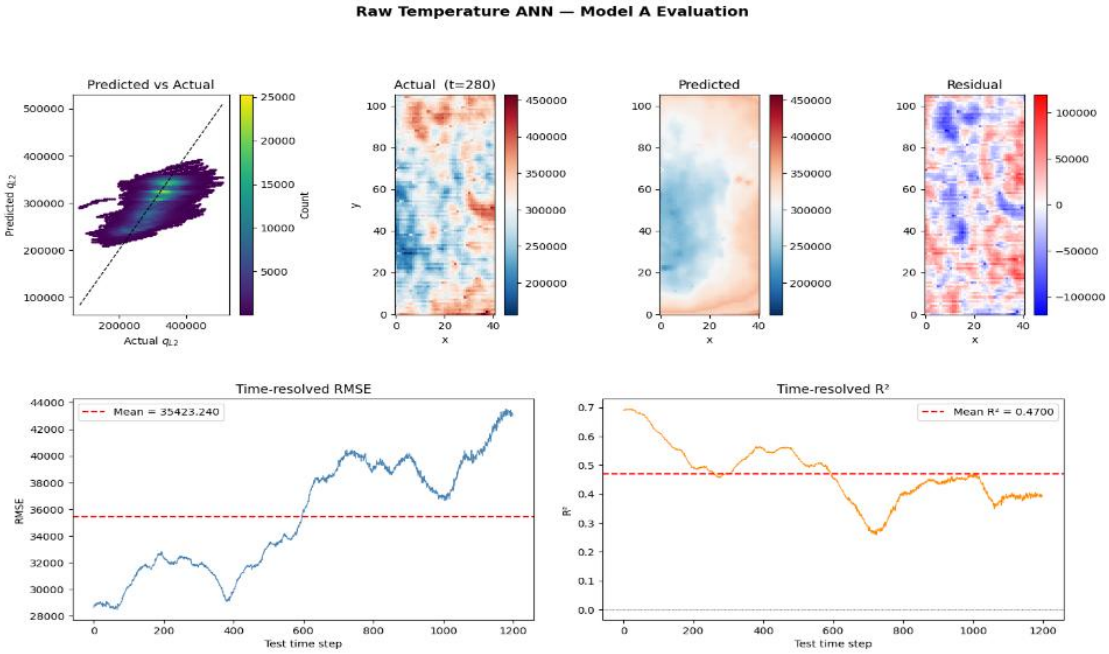


Figure 9 – (left to right) Model A evaluation:(a) predicted versus actual heat flux scatter plot, (b) true heat flux at test timestep 280,(c) predicted heat flux at test time step step 280 (d) residual field. (e) time-resolved RMSE and (f) time resolved R^2

Model A is the simplest model, it uses only one input feature, temperature, to predict heat flux. Hyperparameters are selected through Bayesian optimisation, which has a search space defined in **Error! Reference source not found.** 8.3.5. The model then used these optimal hyperparameters. The model was trained on a million random datapoints in the training-set region, the first 70% of time steps with the last 30% being in the test set, to reduce the computational load. The model went through 200 iterations to converge on a solution.

The Resulting performance was moderate, with test-set metrics that were R^2 = 0.488, $RMSE$ = 35,676.20 $W/m^2$and MAE = 28,200.55 W/m^2 compared with the training metrics of R^2 = 0.476, $RMSE$ = 36,540.51 $W/m^2$and MAE = 29,044.6 W/m^2. Figure 9 show that the model captures the overall relationship between temperature and heat flux, as indicated by the positive trend in the predicted versus actual scatter plot, but the spread around the 1:1 line suggests limited accuracy. The spatial comparison also highlights how the model captures the large-scale structure of the true field, but is more homogeneous and fails to recover the more heterogeneous nature of the true field. Overall, model A provides a useful baseline and demonstrates that temperature contains predictive information.

4.4.2 Model B

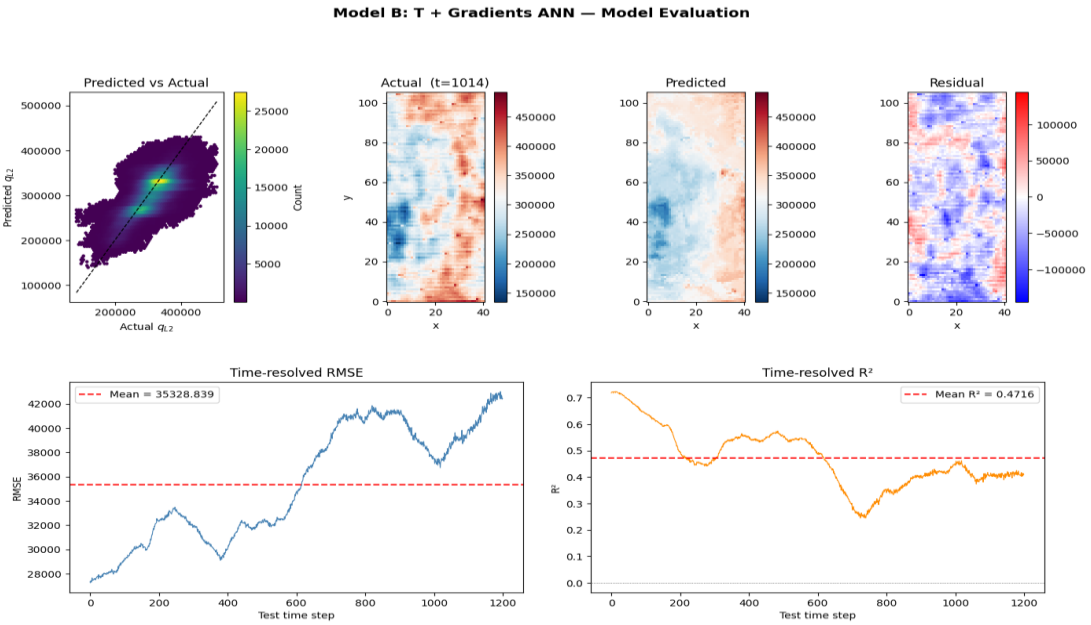


Figure 10 - (left to right) Model B evaluation:(a) predicted versus actual heat flux scatter plot, (b) true heat flux at test timestep 1014,(c) predicted heat flux at test time step 1014,(d) residual field, (e) time-resolved RMSE, and (f) time-resolved R^2

Model B adds to model A by adding spatial and temporal gradients, to capture more spatial information, which, according to the heat equation, should be physically meaningful in predicting heat flux. The input features are $\frac{dT}{dy}$,$\frac{dT}{dx}$, $\frac{dT}{dt}$ and temperature. The same method for training model A was replicated for model B, with the hyperparameters in Appendix 8.3.5

The resulting performance was only slightly better than Model A with test-set metrics was R^2 = 0.4898, $RMSE$ = 35,638.627 $W/m^2$and MAE = 27,791.539 W/m^2 compared with training metrics of R^2 = 0.6036, $RMSE$ = 31,788.537 $W/m^2$and MAE = 24,955.792 W/m^2. Figure 10shows that model B is comparable to model A, as it captures the overall relationship between the inputs and heat flux and like model A, its spatial map captures the broad-scale structure but fails to resolve the finer local heat flux variation. Overall, model B provides only a modest improvement over A, indicating that the addition of gradient features provides only limited improvement in predictive performance.

A notable discrepancy is observed between the training and test performance of Model B, with a train R^2 of 0.604 compared with a test R^2 of 0.490, indicating mild overfitting. Despite the addition of spatial and temporal gradient features, the model learned some relationships in the training data that did not transfer to the test set. This contrasts with Model C, which shows no meaningful train-test gap, suggesting that operating in the reduced POD space naturally limits overfitting by constraining the model to physically dominant structures rather than noise.

4.4.3 Model C

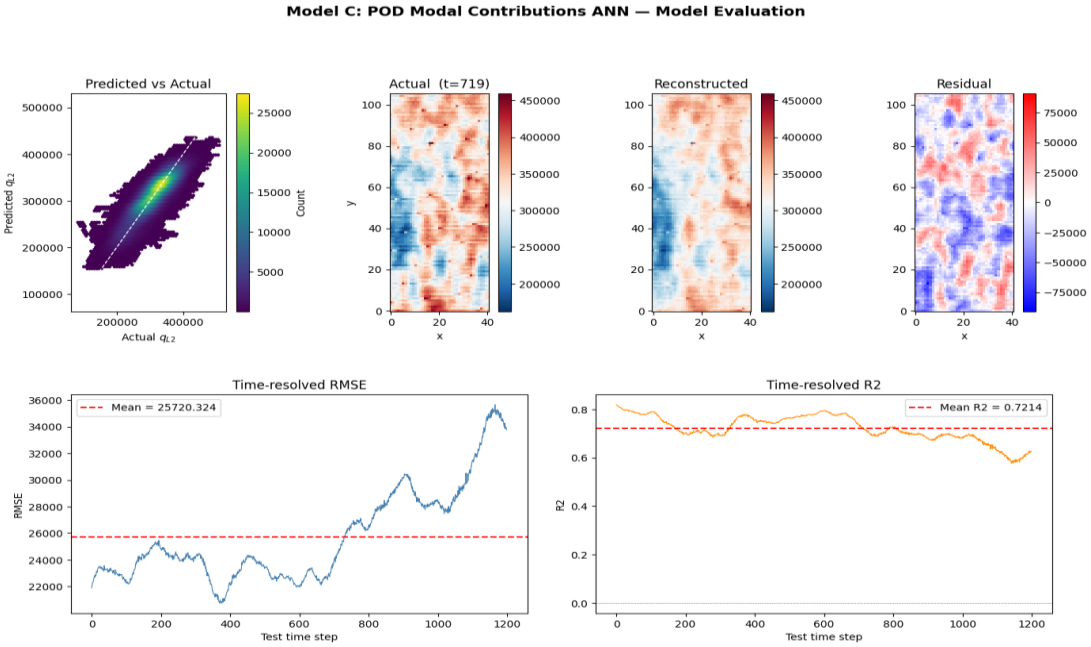


Figure 11 -(left to right) Model C evaluation:(a) predicted versus actual heat flux scatter plot, (b) true heat flux at test timestep719,(c) predicted heat flux at test time step 719,(d) residual field, (e) time-resolved RMSE and (f) time-resolved R^2

Model C is based on POD modal contributions $C_i(x,t)$ .The model's input features are the first five highest energy temperature POD modal contributions. This is then used to predict the top five heat flux modal contributions, which were then used to reconstruct the heat flux field with equation 3. Model C was trained similarly to the previous models; the hyperparameters are in the Appendix 8.3.5

Model C achieved the strongest predictive performance. On the test set R^2 = 0.7293, $RMSE$ = 25,958.517 $W/m^2$and MAE = 20,655.792W/m^2. This represents a substantial improvement over the linear baseline, DMD, and the ANN models based on raw temperature features. This suggests the reduced-order modal representation retains the dominant structure of the temperature heat flux relationship more effectively than the raw-input formulations. The predicted versus actual scatter is tighter around the 1:1 line compared to the previous models, and the reconstructed field preserves the spatial structure of the true heat-flux field more accurately, with lower residual magnitude.

4.5 Comparison and Interpretation

Test set error metrics	Linear baseline (T-q)	DMD	Model A	Model B	Model C
$\mathbf{R^2}$	0.4326*	0.3844	0.4887	0.4898	**0.7293**
RMSE $(\mathbf{W/m^2})$	-	39,144.277	35,676.202	35,638.627	**25,958.517**

MAE (W/m^2)	-	30,525.386	28,200.555	27,791.539	**20,655.792**

Table 1 - Test set error metrics comparison table (over full period)*

Error! Reference source not found. compares the test set performance of the baseline and machine learning models. The simple linear baseline illustrates that temperature explains 43.28% of the variance in the heat flux, indicating that a purely linear relationship between temperature and heat flux is insufficient to describe the data. DMD performed worse in terms of R^2, reflecting its limitation as a longer-horizon reduced-order forecasting method for this problem. Models A and B, both based on ANN regression using raw temperature input with and without gradient features, improved performance approximately $R^2 \approx 0.49$. However, the improvement from Model A to Model B was very small, suggesting that the addition of gradient features alone did not substantially improve predictive accuracy. Model C achieved the strongest overall performance with R^2 = 0.7293, $RMSE$ = 25,958.517 $W/m^2$and MAE = 20,655.792W/m^2. This reveals that POD modal representation provided a more informative and physically meaningful basis for heat flux prediction. The fact that Model C uses only five modes yet outperforms the other models suggests the dominant heat-flux behaviour is governed primarily by low-order structure. However, restricting the model to five modes may still limit its ability to recover local variations in the reconstructed field.

4.6 Limitations

Several limitations should be considered when interpreting the results of this study. First, all three neural network models were trained on a random subsample of one million pixel-time samples rather than the full dataset of 17,384,000 samples, due to computational constraints. While this was sufficient to identify broad patterns, it may have reduced the models' ability to learn rare or localised heat flux behaviour, potentially contributing to the relatively modest R^2 values observed in Models A and B.

Secondly, Model C was restricted to the top five POD modes, which capture 55.3% of the fluctuation energy in the temperature field. Although these dominant modes were sufficient to outperform the raw-input models, the remaining 44.7% of fluctuation energy representing finer-scale transient behaviour was discarded. It is possible that retaining a greater number of modes would improve reconstruction accuracy, particularly in regions of high local heat flux variation.

Thirdly, the dataset originates from a single flow boiling experiment conducted under specific operating conditions. The generalisability of the trained models to different fluids, heater geometries, heat flux levels, or flow conditions has not been assessed. This limits the transferability of the methodology beyond the present dataset.

Lastly, the temporal ordourering of the train-test split means the test set corresponds to a later portion of the boiling process, during which the mean heat flux is slightly higher (313,161 W/m^2versus 302,262W/m^2). This distributional shift makes the test problem marginally

more challenging than the training problem and may result in a modest underestimate of model performance under matched conditions.

5 Conclusion and Future Work

This project investigated whether machine learning models could predict local boiling heat flux from transient infrared temperature data, improving on traditional empirical correlations limited to averaged, steady-state behaviour.

Three models of increasing complexity were developed and evaluated. Model A used raw temperature alone and achieved R^2 = 0.489, confirming that temperature is physically informative but insufficient on its own. Model B added spatial and temporal gradients but provided only marginal improvement (R^2 = 0.490) with signs of mild overfitting, indicating the additional features did not substantially enrich the representation. Model C mapped the top five temperature POD modal contributions to their heat flux equivalents, achieving a test R^2= 0.729, a 69% improvement over the linear baseline. This was supported by the high correlation between temperature and heat flux temporal coefficients(r ≥ 0.96 for three of five modes), providing physical justification for the modal mapping strategy.

The results demonstrate that operating in a reduced POD space offers a more effective basis for heat flux prediction than raw input formulations, resolving transient boiling behaviour beyond the capability of conventional correlations.

Computational constraints limited training to a dataset subsample and restricted Model C to five POD modes, capturing 55% of the fluctuation energy. The single-experiment dataset also limits generalisability to other fluids, geometries or operating conditions. Future work should examine the effect of retaining more modes, evaluate the approach across wider boiling conditions, and investigate improved regularisation to reduce the train-test gap observed in Model B.

6 Acknowledgements

The author would like to express sincere gratitude to Dr Huayong Zhao for his supervision and guidance, and continued support throughout this project. Appreciation is also extended to Dr Chow Yin Lai for his assistance with the machine learning model construction. Finally, the author would like to thank Professor J. Nathan Kutz, whose publicly available educational resources helped demystify the POD and DMD techniques central to this work.

7 References

[1] A. Upadhyay, S. K. Hazra, A. Assam, and R. Raj, 'Review of the current status and the potential of machine learning tools in boiling heat transfer', *Numerical Heat Transfer, Part B: Fundamentals*, vol. 85, no. 11, pp. 1489–1532, 2024, doi: 10.1080/10407790.2023.2266770.

[2] J. H. Lienhard, 'A Heat Transfer Textbook, 6th edition', Apr. 2024.

[3] S. Nukiyama, 'The Maximum and Minimum Values of The Heat Q transmitted from Metal to Boiling water under atmospheric pressure', 1934.

[4] R. W. M., 'A Method of Correlating Heat-Transfer Data for Surface Boiling of Liquids', 1951. Accessed: Mar. 30, 2026. [Online]. Available: https://watermark02.silverchair.com/

[5] S. G. Kandlikar, 'A General Correlation for Saturated Two-Phase Flow Boiling Heat Transfer Inside Horizontal and Vertical Tubes', 1990. [Online]. Available: http://asmedigitalcollection.asme.org/heattransfer/article-pdf/112/1/219/5644274/219_1.pdf

[6] N. Zuber, 'HYDRODYNAMIC ASPECTS OF BOILING HEAT TRANS FER', University of California , Los Angeles, California , 1959. Accessed: Mar. 31, 2026. [Online]. Available: https://www.osti.gov/servlets/purl/4175511

[7] H. M. Ertunc, 'Prediction of the Pool Boiling Critical Heat Flux Using Artificial Neural Network', 2006. Accessed: Apr. 01, 2026. [Online]. Available: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=4016219

[8] A. S. Surtaev, V. S. Serdyukov, and M. I. Moiseev, 'Application of high-speed IR thermography to study boiling of liquids', *Instruments and Experimental Techniques*, vol. 59, no. 4, pp. 615–620, Jul. 2016, doi: 10.1134/S0020441216030222.

[9] H. Chu, T. Ji, X. Yu, Z. Liu, Z. Rui, and N. Xu, 'Advances in the application of machine learning to boiling heat transfer: A review ', 2024. Accessed: Apr. 07, 2026. [Online]. Available: https://pdf.sciencedirectassets.com/

[10] B. Ma, Z. Zhao, M. Sun, and B. Liu, 'Machine learning-based prediction of nucleate pool boiling heat transfer enhancement on micropillar surfaces', *International Communications in Heat and Mass Transfer*, vol. 166, p. 109116, Aug. 2025, doi: 10.1016/j.icheatmasstransfer.2025.109116.

[11] A. Hajisharifi, R. Halder, M. Girfoglio, G. Stabile, and G. Rozza, 'A Deep-Learning Enhanced Gappy Proper Orthogonal Decomposition Method for Conjugate Heat Transfer Problem', Aug. 2025, [Online]. Available: http://arxiv.org/abs/2508.20633

8 Appendix

8.1 MATLAB Code

8.1.1 Data preprocessing

```
z1 = 1;   % nz

T_z1 = squeeze(T(:, :,z1 ,:));  % [nx, ny, nt]
crop = 5;   % adjust this later

T_crop   = T_top(1+crop:end-crop, 1+crop:end-crop, :);
qL2_crop = qL2(  1+crop:end-crop, 1+crop:end-crop, :);
```

8.1.2 Preparing for SVD

```
[nx, ny, nt] = size(T_crop);

Npix = nx * ny;

% Reshape T_crop into X: [Npix × nt]
X = reshape(T_crop, [Npix, nt]);

% Convert each column to have zero mean (POD requires this)
X_mean = mean(X, 2);    % spatial mean over time
Xc = X - X_mean;        % subtract mean from each column
```

8.1.3 SVD, temporal coeffiecents and energy computation

```
[U, S, V] = svd(Xc, 'econ');   % U: modes, S: singular values, V: time
coeffs

sig = diag(S);           % singular values
energy = sig.^2 / sum(sig.^2);

% Compute cumulative energy
cum_energy = cumsum(energy);

A = S * V';   % A(k,t) = amplitude of mode k at time t
```

8.1.4 Plotting spatial modes and temporal coefficients

```
% ---- Plot spatial modes
for k = 1:nmodes_to_plot
    mode_k = reshape(U(:,k), [nx, ny]);

    % Keep subplot layout fixed and ensure each image uses the same axes
size
    ax = subplot(2, nmodes_to_plot, nmodes_to_plot + k);
    imagesc(ax, mode_k); axis(ax, 'image');

    % Symmetric colour limits about zero => sign is meaningful
    if use_global_clim
        clim = global_clim;
    else
```

```
        clim = max(abs(mode_k(:)));
    end
    caxis(ax, [-clim, +clim]);

    colormap(ax, bluewhitered());  % diverging colormap (defined below)
    cb = colorbar(ax);
    cb.Label.String = 'Mode amplitude (signed)';
    title(ax, sprintf('POD Mode %d (Energy = %.2f%%)', k, energy(k)*100));
    xlabel(ax, 'y index'); ylabel(ax, 'x index');

   ;
end

sgtitle('Mean field and signed POD modes (diverging colourmap, symmetric
limits)');

% ---- Plot Temporal amplitudes

modes_to_show = min(5, size(A,1));         % plot first 5 by default
nplot = min(nt, 4043);                     % don't over-plot if nt is huge
tt = 1:nplot;

figure('Name','Temporal coefficients');
for k = 1:modes_to_show
    subplot(modes_to_show, 1, k);
    plot(tt, A(k, tt), 'LineWidth', 1.2);
    grid on;
    ylabel(sprintf('a_%d', k));
    if k == 1
        title('Temporal coefficients a_k(t) (amplitude of each POD mode)');
    end
    if k == modes_to_show
        xlabel('Time index');
    end
end
```

8.1.5 DMD Prediction

```
%% ============================================================
% DMD ON HEAT FLUX — proper forecast + error metrics
% Forecast is initialised from the last training frame.
% ============================================================
%% SECTION 2 — crop boundaries
crop = 5;
Q_crop = qL2(1+crop:end-crop, 1+crop:end-crop, :);
time_crop = 43;
Q_crop = Q_crop(:,:,1:end-time_crop);
%% SECTION 3 — time vector
[nx, ny, nt] = size(Q_crop);
if numel(TimeStep) > 1
t = TimeStep(:);
else
dt = TimeStep;
t = (0:nt-1)' * dt;
end
dt = t(2) - t(1);
Npix = nx * ny;
disp(size(Q_crop)); disp(ndims(Q_crop));
fprintf('Loaded: nx=%d, ny=%d, nt=%d, Npix=%d, dt=%.6g s\n', nx, ny, nt,
Npix, dt);
```

```
%% SECTION 4 — snapshot matrix
X = reshape(Q_crop, [Npix, nt]);
%% SECTION 5 — user settings
train_frac = 0.7; % fraction of nt used for training
trainN = round(train_frac * nt); % 70% of available frames
assert(trainN <= nt, 'Not enough frames for trainN.');
test_start = trainN + 1;
test_end = nt;
compare_end = nt;
fprintf('Train: 1..%d (%.0f%% of %d) | Forecast: %d..%d\n', ...
trainN, train_frac*100, nt, test_start, test_end);
%% SECTION 6 — build DMD model from training window
X_mean = mean(X(:,1:trainN), 2);
Xc = X - X_mean;
X1 = Xc(:, 1:trainN-1);
X2 = Xc(:, 2:trainN);
[U, S, V] = svd(X1, 'econ');
singvals = diag(S);
energy_svd = cumsum(singvals.^2) / sum(singvals.^2);
r = find(energy_svd >= 0.99, 1);
fprintf('r at 99%% energy = %d\n', r);
Ur = U(:,1:r); Sr = S(1:r,1:r); Vr = V(:,1:r);
%% SECTION 7 — reduced DMD operator and eigen-decomposition
Atilde = Ur' * X2 * Vr / Sr;
[W, D] = eig(Atilde);
lambda = diag(D);
Phi = X2 * Vr / Sr * W;
%% SECTION 8 — continuous-time rates
omega = log(lambda) / dt;
%% ============================================================
% SECTION 9 — TWO PREDICTIONS using the same linear operator
% (a) Training prediction: initialised from frame 1, covers 1..trainN
% (b) Forecast: initialised from frame trainN, covers trainN..end
% ============================================================
% --- (a) Training prediction from frame 1 ------------------
b_tr = Phi \ Xc(:,1);
k_tr = 0:(trainN-1);
X_tr_local = real(Phi * (b_tr .* exp(omega * k_tr * dt))) + X_mean;
X_tr = nan(Npix, compare_end);
X_tr(:, 1:trainN) = X_tr_local;
% --- (b) Forecast from last training frame -----------------
b_fc = Phi \ Xc(:, trainN);
nfc = compare_end - trainN + 1;
k_fc = 0:(nfc-1);
X_fc_local = real(Phi * (b_fc .* exp(omega * k_fc * dt))) + X_mean;
X_fc = nan(Npix, compare_end);
X_fc(:, trainN:end) = X_fc_local;
```

8.2 python code

8.2.1 feature construction modal contributions

```
import numpy as np


n_modes_feat = 5

nx, ny, nt = T_crop.shape
```

```
# ── Temperature POD ──────────────────────────────────────────────────────
X = T_crop.reshape(nx * ny, nt)
X_mean = np.mean(X, axis=1, keepdims=True)

train_function = 0.7
nt_train_svd = int(nt * train_function)

Xc_train = X[:, :nt_train_svd] - X_mean
Xc_test  = X[:, nt_train_svd:] - X_mean

U, S, VT_train = np.linalg.svd(Xc_train, full_matrices=False)
VT_test = (U[:, :n_modes_feat].T @ Xc_test) / S[:n_modes_feat, np.newaxis]
VT = np.concatenate([VT_train[:n_modes_feat, :], VT_test], axis=1)

energy_all = S**2
cum_energy_all = np.cumsum(energy_all) / np.sum(energy_all)
print(f'T energy captured by first {n_modes_feat} modes:
{cum_energy_all[n_modes_feat-1]:.2%}')

mode_contribs = np.zeros((nx, ny, nt, n_modes_feat))
for i in range(n_modes_feat):
    phi_i = U[:, i]
    a_i   = S[i] * VT[i, :]
    Ci    = np.outer(phi_i, a_i)
    mode_contribs[:, :, :, i] = Ci.reshape(nx, ny, nt)
print('T mode_contribs shape:', mode_contribs.shape)

# ── qL2 POD ──────────────────────────────────────────────────────────────
Q = qL2_crop.reshape(nx * ny, nt)
Q_mean = np.mean(Q, axis=1, keepdims=True)

Qc_train = Q[:, :nt_train_svd] - Q_mean
```

```
Qc_test  = Q[:, nt_train_svd:] - Q_mean


U_q, S_q, VT_q_train = np.linalg.svd(Qc_train, full_matrices=False)

VT_q_test = (U_q[:, :n_modes_feat].T @ Qc_test) / S_q[:n_modes_feat,
np.newaxis]

VT_q = np.concatenate([VT_q_train[:n_modes_feat, :], VT_q_test], axis=1)


energy_q = S_q**2

cum_energy_q = np.cumsum(energy_q) / np.sum(energy_q)

print(f'qL2 energy captured by first {n_modes_feat} modes:
{cum_energy_q[n_modes_feat-1]:.2%}')


qL2_mode_contribs = np.zeros((nx, ny, nt, n_modes_feat))

for i in range(n_modes_feat):

    phi_i = U_q[:, i]

    a_i   = S_q[i] * VT_q[i, :]

    Ci    = np.outer(phi_i, a_i)

    qL2_mode_contribs[:, :, :, i] = Ci.reshape(nx, ny, nt)

print('qL2 mode_contribs shape:', qL2_mode_contribs.shape)
```

8.2.2 feature construction temperature and spatial gradients

```
dt = np.mean(np.diff(TimeStep.squeeze()))

dTdt = np.gradient(T_rec, dt, axis=2)   # temporal

dTdx = np.gradient(T_rec, axis=1)       # spatial x

dTdy = np.gradient(T_rec, axis=0)       # spatial y
```

8.2.3 Bayesian optimisation (Model C)

```
import optuna

from sklearn.model_selection import train_test_split

optuna.logging.set_verbosity(optuna.logging.WARNING)


# Smaller subset for fast trials

N_OPT = 200_000

rng_opt = np.random.default_rng(0)

idx_opt = rng_opt.choice(len(X_sub), size=N_OPT, replace=False)

X_opt = X_sub[idx_opt]
```

```
y_opt = y_sub[idx_opt]

# 80/20 validation split used inside every trial
X_tr, X_val, y_tr, y_val = train_test_split(
    X_opt, y_opt, test_size=0.2, random_state=42
)

def objective(trial):
    n_layers   = trial.suggest_int('n_layers', 1, 3)
    n_units    = trial.suggest_int('n_units', 16, 256, step=16)
    activation = trial.suggest_categorical('activation', ['relu', 'tanh'])
    alpha      = trial.suggest_float('alpha', 1e-5, 1e-1, log=True)
    lr_init    = trial.suggest_float('lr_init', 1e-4, 1e-2, log=True)

    mlp = MLPRegressor(
        hidden_layer_sizes=tuple([n_units] * n_layers),
        activation=activation,
        solver='adam',
        alpha=alpha,
        learning_rate_init=lr_init,
        max_iter=200,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=10,
        random_state=42,
    )
    mlp.fit(X_tr, y_tr)
    return r2_score(y_val, mlp.predict(X_val),
multioutput='uniform_average')

study = optuna.create_study(direction='maximize',
                            sampler=optuna.samplers.TPESampler(seed=42))
study.optimize(objective, n_trials=30, show_progress_bar=True)
```

```
best = study.best_params
print('\n=== BEST HYPERPARAMETERS ===')
for k, v in best.items():
    print(f'  {k}: {v}')
print(f'  validation R²: {study.best_value:.4f}')
```

8.2.4 Machine Learning Algorithm (ModelC)

```
best_layers = tuple([best['n_units']] * best['n_layers'])

model = MLPRegressor(
    hidden_layer_sizes=best_layers,
    activation=best['activation'],
    solver='adam',
    alpha=best['alpha'],
    learning_rate_init=best['lr_init'],
    max_iter=200,
    early_stopping=True,
    validation_fraction=0.1,
    n_iter_no_change=10,
    random_state=42,
)

model.fit(X_sub, y_sub)
print('Final model trained with:', best_layers, best['activation'])
```

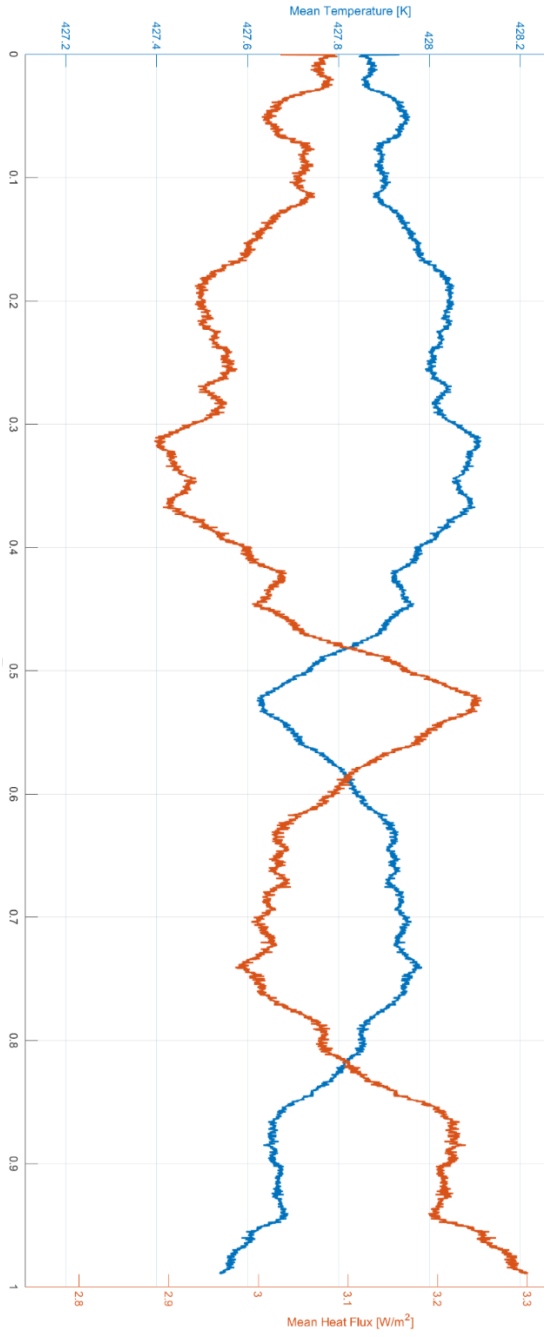
8.2.5 Error Metrics

```
# ------------------------------------------------------------
# Predict
# ------------------------------------------------------------
y_train_pred_scaled = model.predict(X_train_scaled)
y_test_pred_scaled = model.predict(X_test_scaled)

# Convert predictions back to original qL2 scale
```

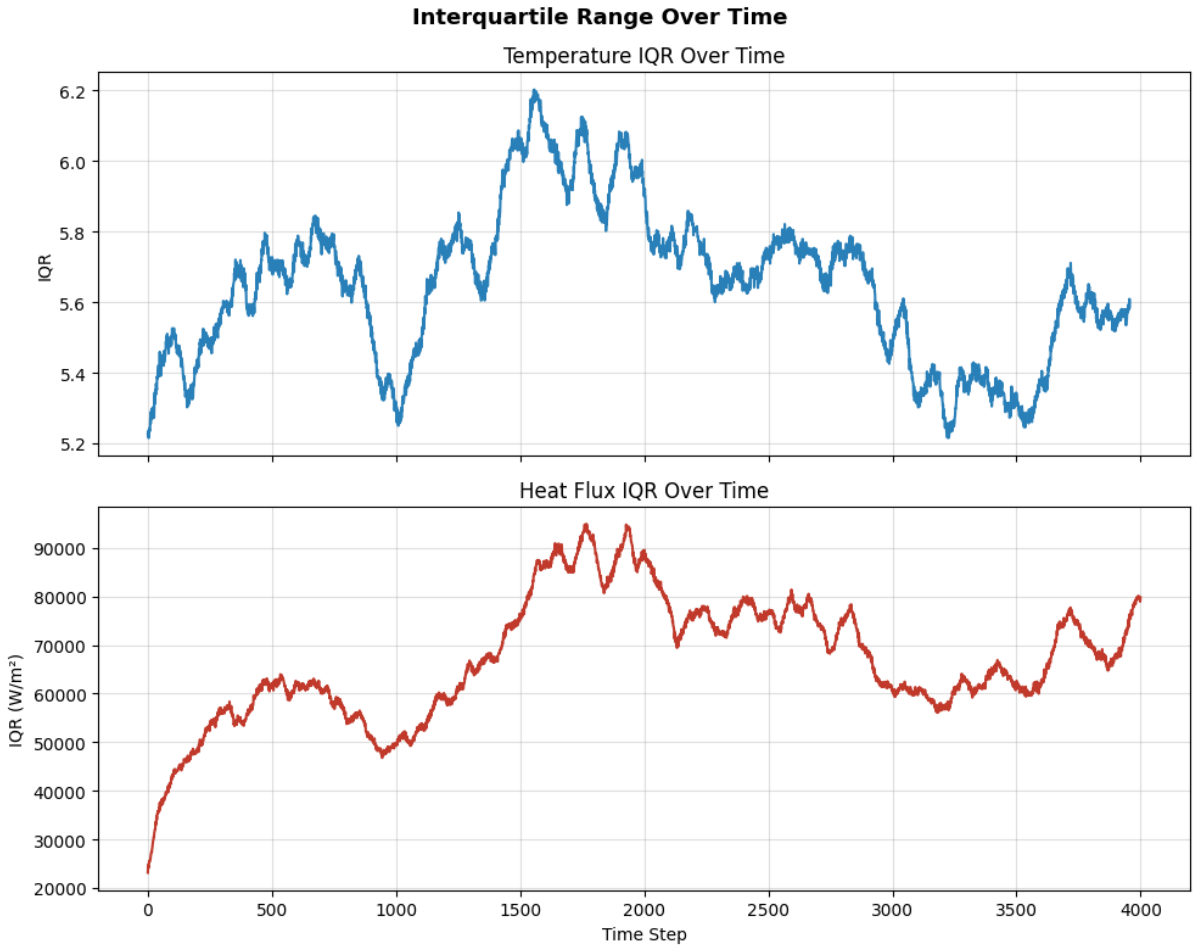
```
y_train_pred = scaler_y.inverse_transform(y_train_pred_scaled.reshape(-1,
1)).ravel()
y_test_pred = scaler_y.inverse_transform(y_test_pred_scaled.reshape(-1,
1)).ravel()


# ------------------------------------------------------------
# Metrics
# ------------------------------------------------------------
rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred))
mae_train = mean_absolute_error(y_train, y_train_pred)
r2_train = r2_score(y_train, y_train_pred)

rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))
mae_test = mean_absolute_error(y_test, y_test_pred)
r2_test = r2_score(y_test, y_test_pred)

print("=== TRAIN ===")
print("RMSE:", rmse_train)
print("MAE :", mae_train)
print("R^2 :", r2_train)

print("\n=== TEST ===")
print("RMSE:", rmse_test)
print("MAE :", mae_test)
print("R^2 :", r2_test)
```

8.3 Graphs and Tables

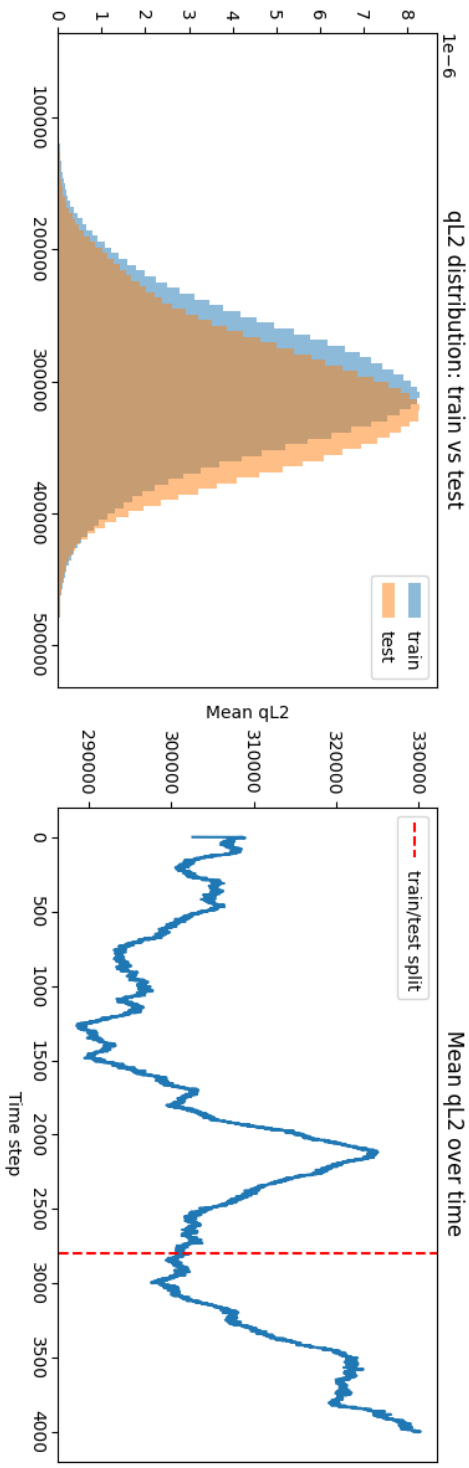
8.3.1 mean heat flux and mean surface temperature overlay



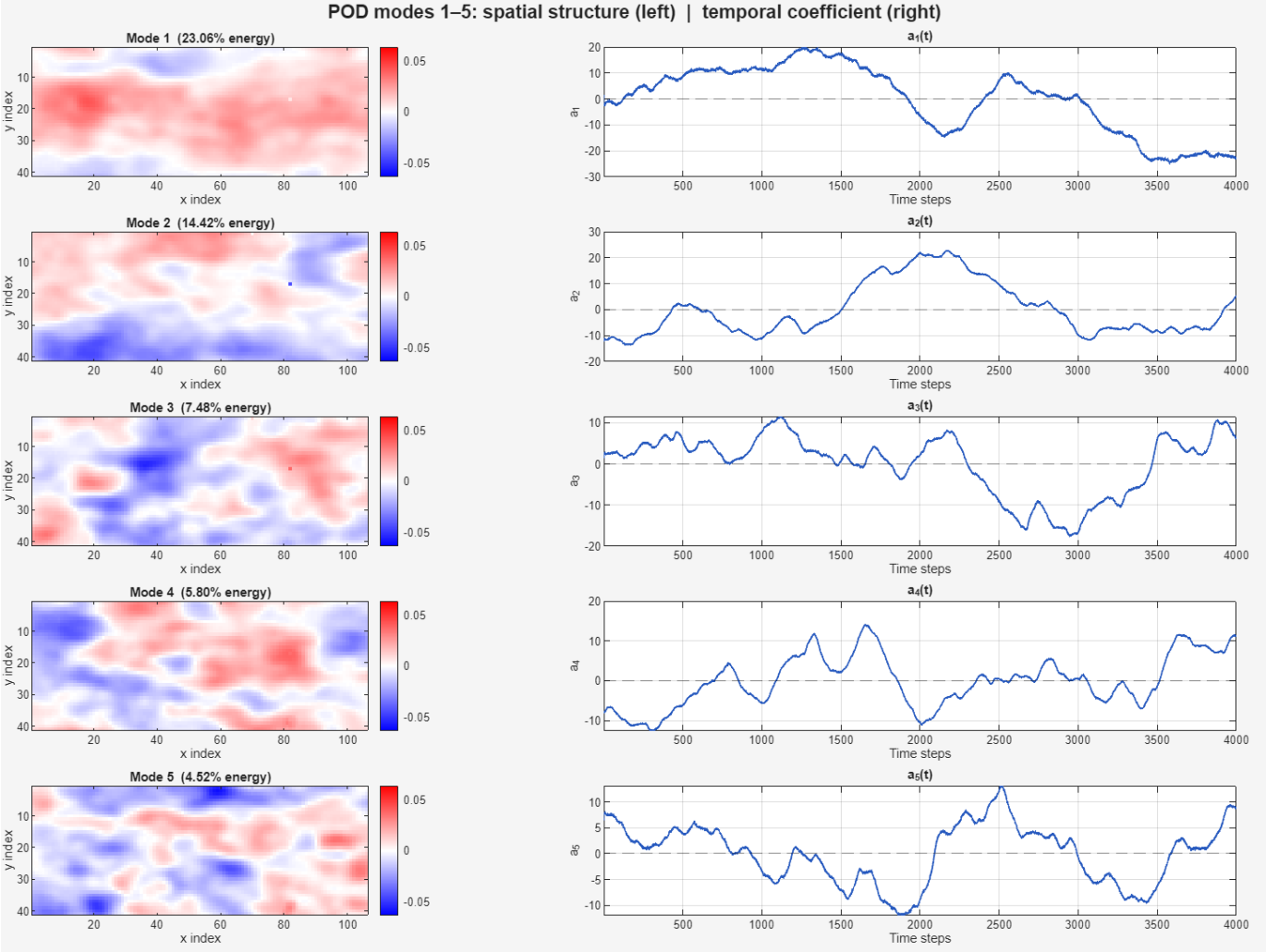
8.3.2 IQR of Heat flux and Temperature

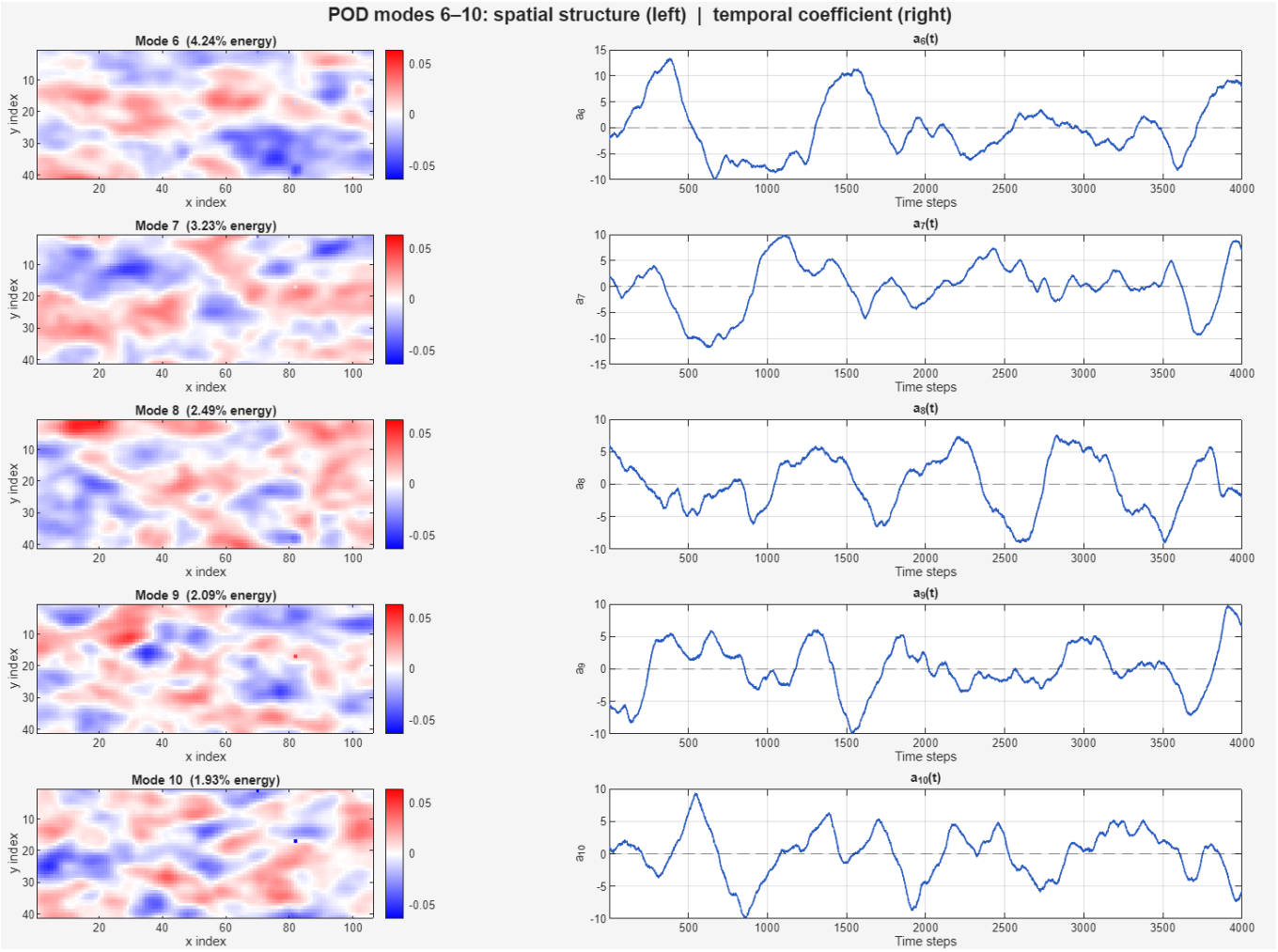


8.3.3 Heat flux training vs test set comparison



8.3.4 Temperature POD Modes and temporal coefficients




POD modes 6–10: spatial structure (left) | temporal coefficient (right)
Mode 6 (4.24% energy)
Mode 7 (3.23% energy)
Mode 8 (2.49% energy)
Mode 9 (2.09% energy)
Mode 10 (1.93% energy)
x index
y index
a6(t)
a7(t)
a8(t)
a9(t)
a10(t)
Time steps

8.3.5 Bayesian optimisation hyperparameters

Model A

Hyperparameters	Search Space	Optimal Hyperparameters
Neurons per layer	4,8,12,16,20,24,28,32	32
Hidden layers	1,2	2
Activation	Relu, Tanh	Relu
Alpha	1e-5, 1e-1	0.005158752049482879
Learning rate	1e-4, 1e-2	0.0001619808795852012

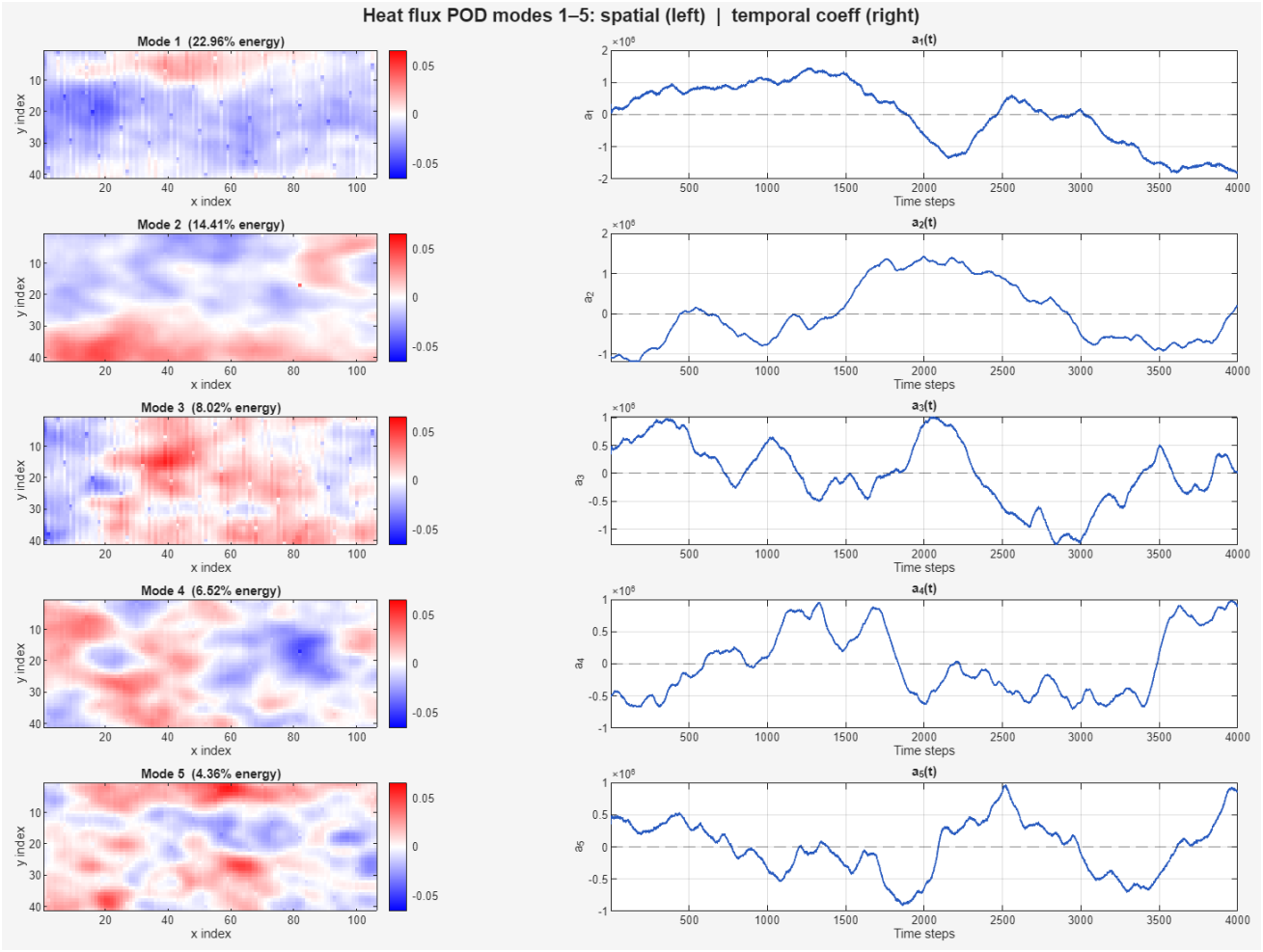
Model B

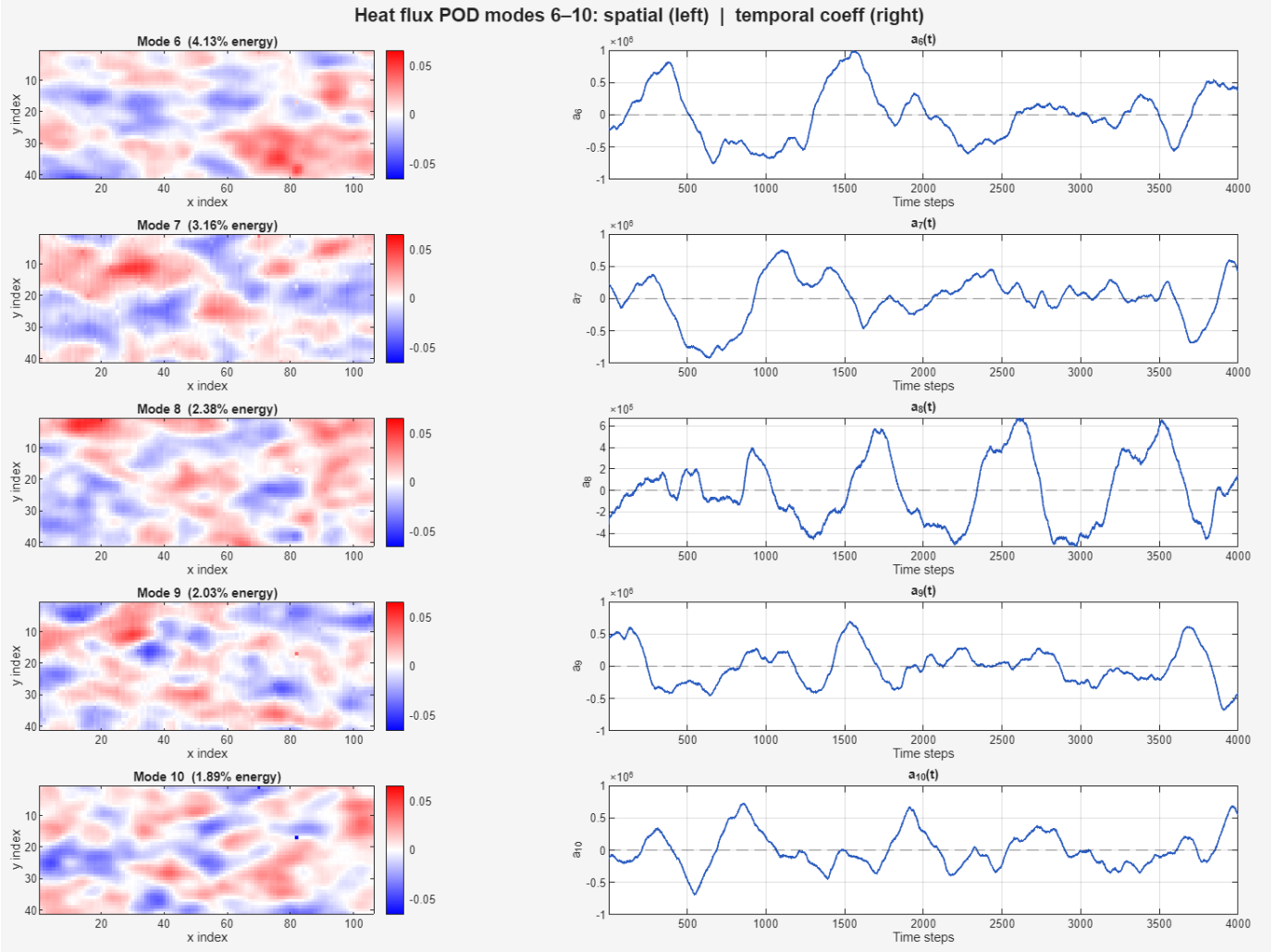
Hyperparameters	Search Space	Optimal Hyperparameters
Neurons per layer	8,16,24,32,40,48,56,64	64
Hidden layers	1,3	3
Activation	Relu, Tanh	Tanh
Alpha	1e-5, 1e-1	0.0012893538030423208
Learning rate	1e-4, 1e-2	0.0015151645560230295

Model C

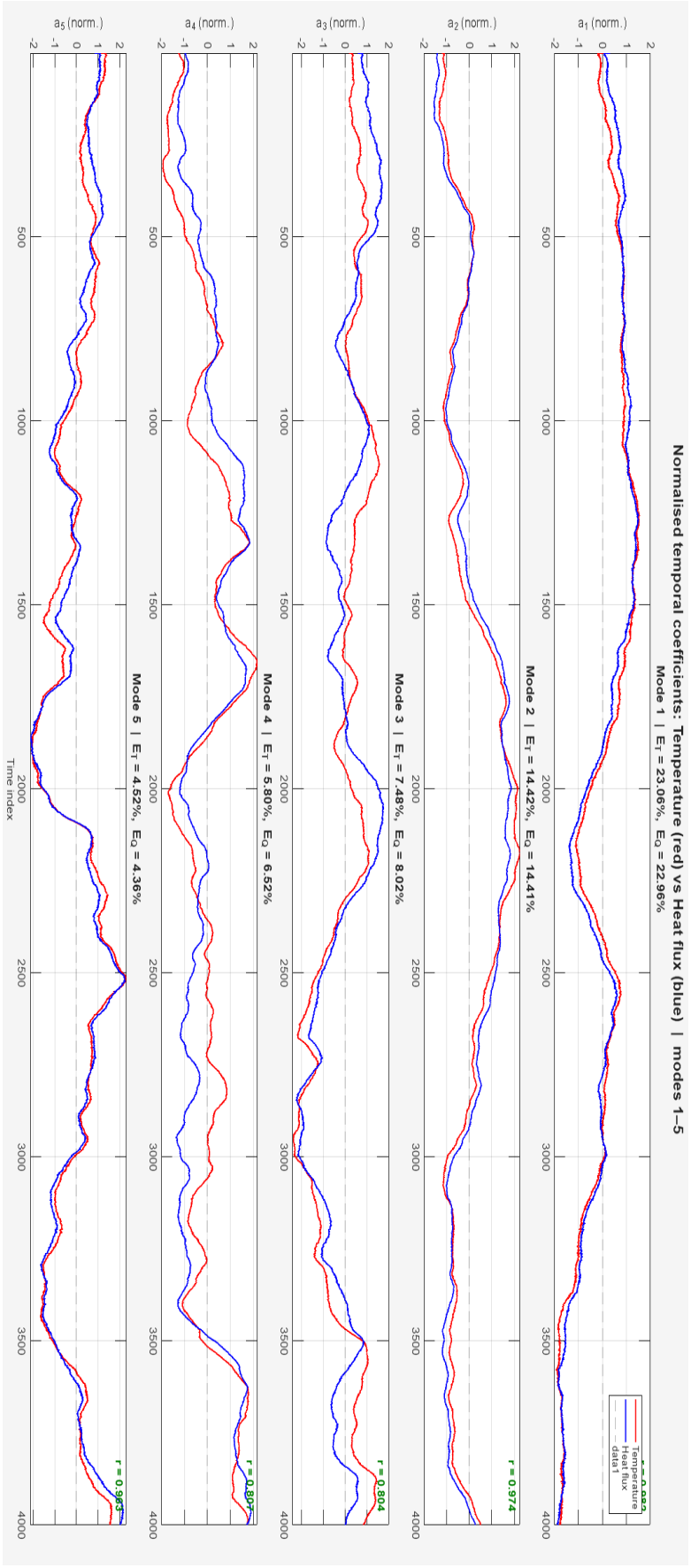
Hyperparameters	Search Space	Optimal Hyperparameters
Neurons per layer	16: 16 : 256	192
Hidden layers	1,3	3
Activation	Relu, Tanh	Tanh
Alpha	1e-5, 1e-1	0.0006312862053858106
Learning rate	1e-4, 1e-2	0.0006793020634935387

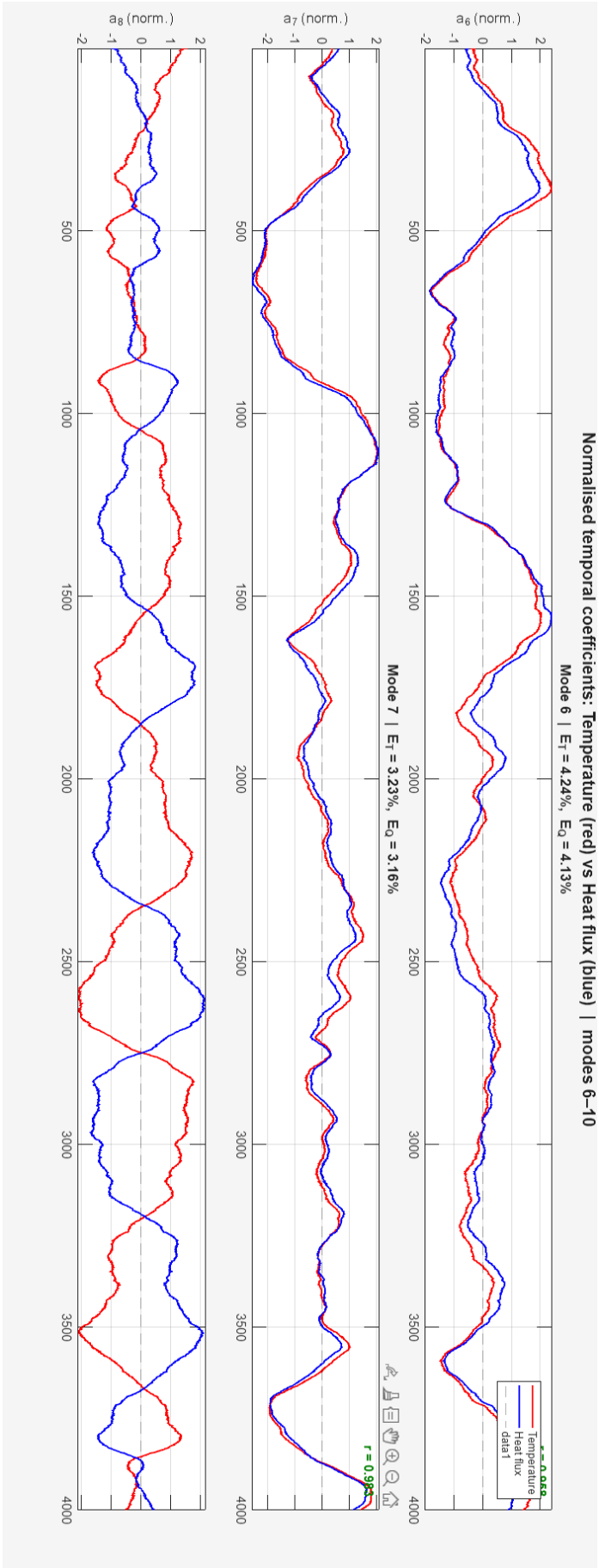
8.3.6 Heat flux POD Modes and temporal coefficients



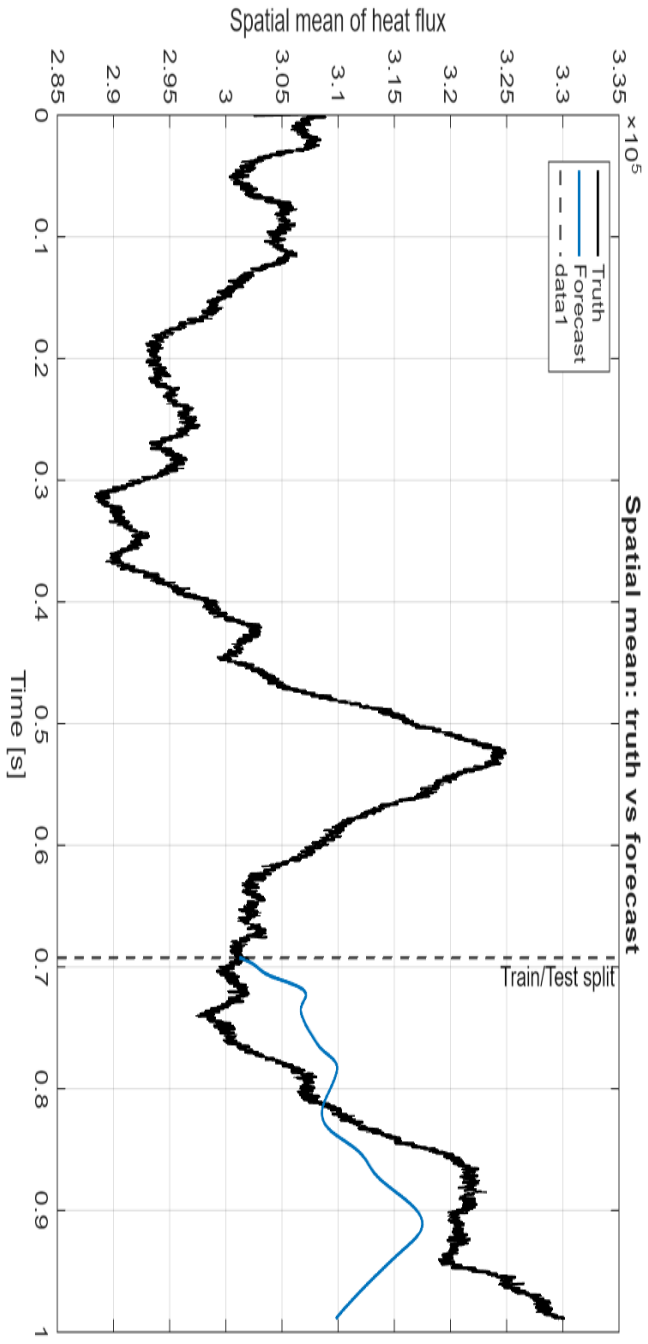

Heat flux POD modes 6–10: spatial (left) | temporal coeff (right)
Mode 6 (4.13% energy)
Mode 7 (3.16% energy)
Mode 8 (2.38% energy)
Mode 9 (2.03% energy)
Mode 10 (1.89% energy)
x index
y index
Time steps
a6(t)
a7(t)
a8(t)
a9(t)
a10(t)

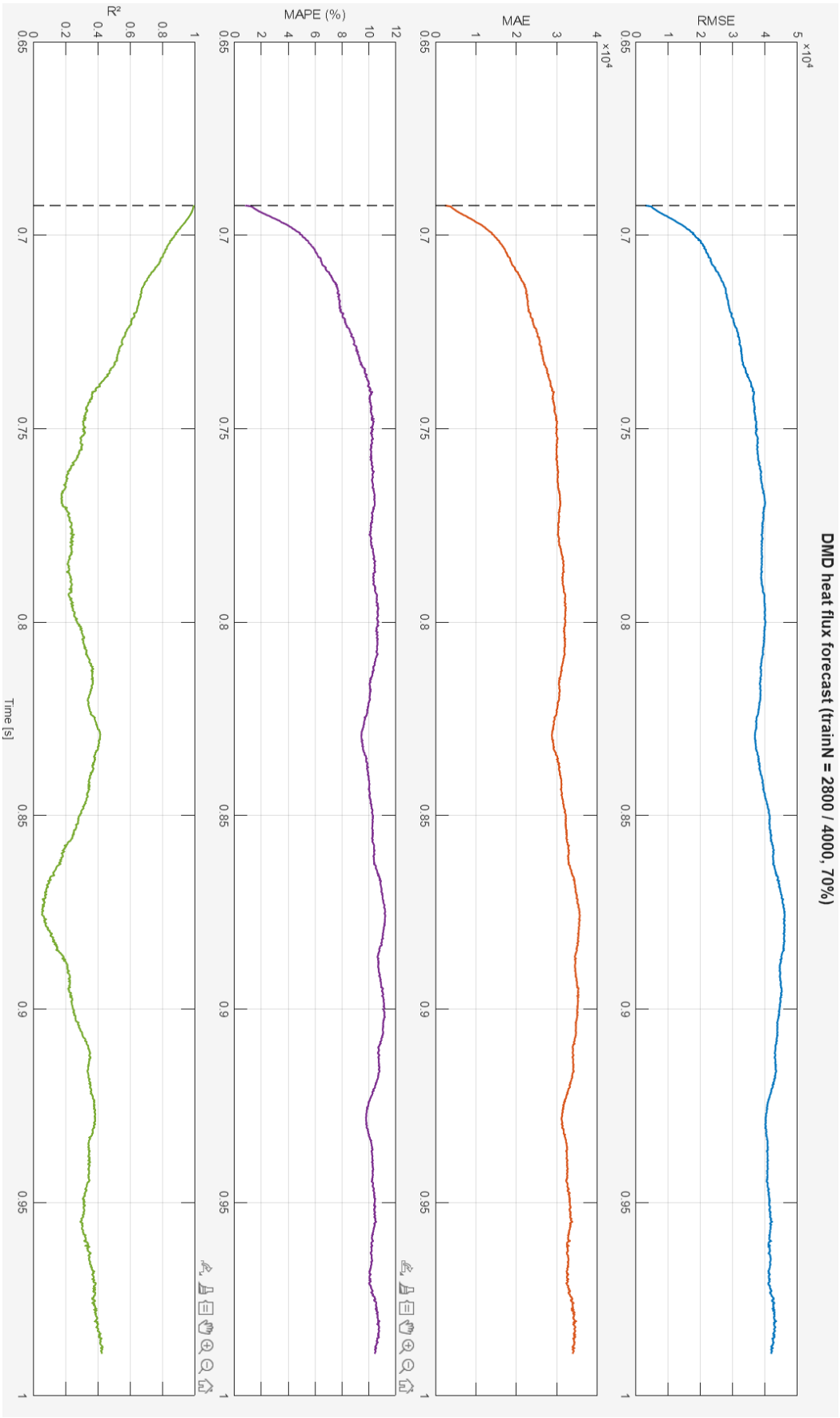
8.3.7 Heat flux and temperature temporal coefficients comparison




Normalised temporal coefficients: Temperature (red) vs Heat flux (blue) | modes 6–10
Mode 6 | E_T = 4.24%, E_Q = 4.13%
Mode 7 | E_T = 3.23%, E_Q = 3.16%
a_6 (norm.)
a_7 (norm.)
a_8 (norm.)
Temperature
Heat flux
data1
r = 0.984

8.3.8 DMD forecast error metrics




DMD heat flux forecast (trainN = 2800 / 4000, 70%)
RMSE
MAE
MAPE (%)
R²
Time [s]

8.4 Schematics

8.4.1 Model C Schematic

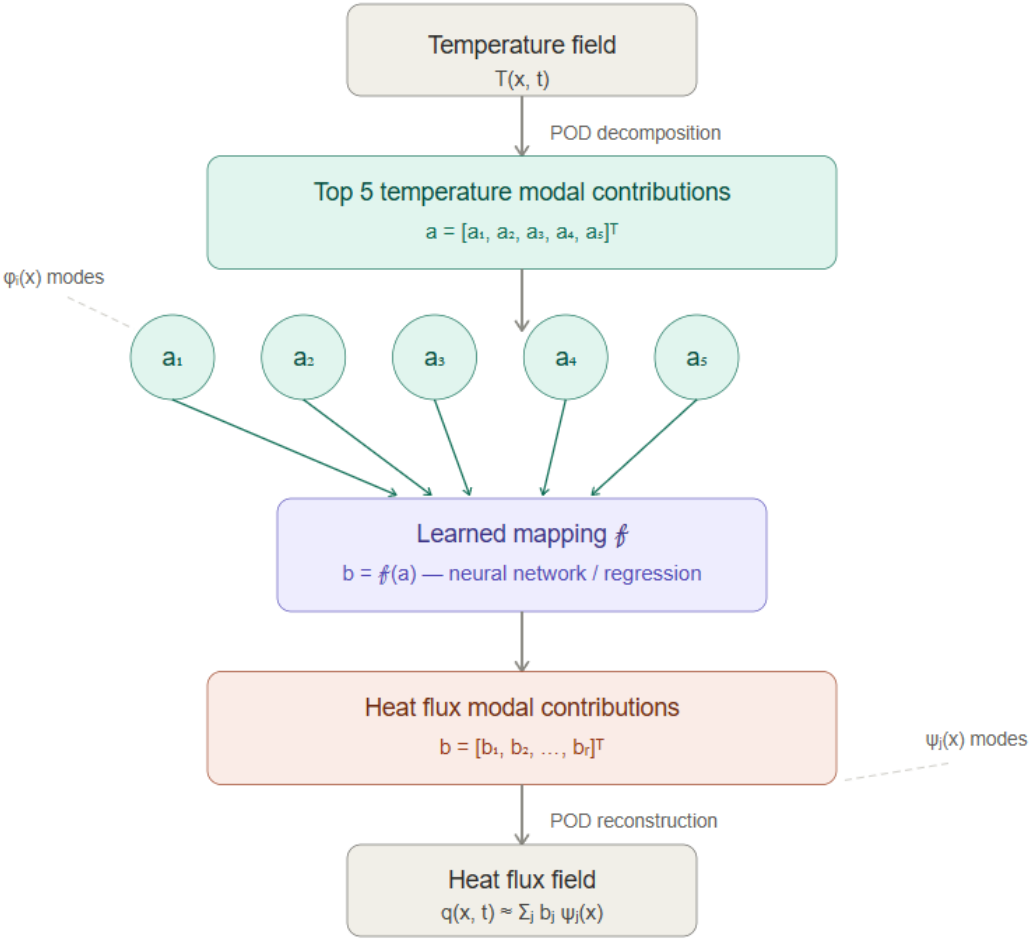


Figure 12 – Model C schematic [Ai generated image by Claude sonnet]

8.4.2 Temperature dataset schematic

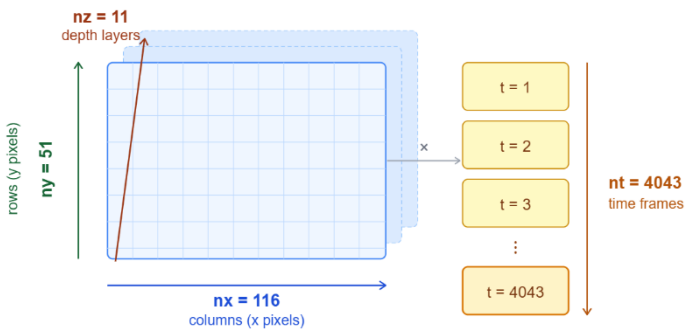


Figure 13 - Temperature dataset schematic [AI generated image by claude sonnet]

8.4.3 DMD Schematic